



Universidad Politécnica
de Madrid



**Escuela Técnica Superior de
Ingenieros Informáticos**

Grado en Ingeniería Informática

Memoria Final

**Desarrollo de una Aplicación para la
Planificación y Gestión de Viajes en
Grupo**

Autor: Andrés García Solórzano

Tutor: Antonio Jesús Díaz Honrubia

Madrid, junio 2023

Este Trabajo Fin de Grado se ha depositado en la ETSI Informáticos de la Universidad Politécnica de Madrid para su defensa.

Trabajo Fin de Grado

Grado en Ingeniería Informática

Título: Desarrollo de una Aplicación para la Planificación y Gestión de Viajes en Grupo

Junio 2023

Autor: Andrés García Solórzano

Tutor: Antonio Jesús Díaz Honrubia

Departamento de Lenguajes y Sistemas Informáticos e Ingeniería del Software

ETSI Informáticos

Universidad Politécnica de Madrid

Resumen

Este TFG consiste en el desarrollado de una aplicación que permite a los usuarios planificar y gestionar sus viajes de forma sencilla. El funcionamiento de ésta es el siguiente. Después de iniciar sesión los usuarios se encuentran con una pantalla que muestra de varias sus viajes futuros y pasados y un calendario con todas las fechas. Desde esta pantalla el usuario se puede mover moverse a la pantalla específica de cada viaje.

En la pantalla de información de cada viaje se muestra el nombre y estado de planificación de este, una lista de tarjetas con información relevante sobre el viaje (tarjetas de componente) y dos botones, uno para modificar información del viaje e invitar o eliminar participantes y otro para añadir nuevos componentes.

La aplicación cuenta con 7 tipos de componente distintos, cada uno para un aspecto de la planificación del viaje. Cada tipo de componente tiene dos representaciones visuales, la primera es la tarjeta del componente que actúa como vista previa permitiendo al usuario ver un resumen de su contenido y que en caso de ser pulsada lleva al usuario a la segunda representación del componente, una pantalla en la que se muestra toda la información del y permite al usuario actualizarla en tiempo real.

Los 7 tipos de componente que se pueden añadir a un viaje son:

- Componente de distribución de habitaciones
- Componente de confirmación de asistencia
- Componente de gestión de gastos
- Componente de lista de la compra
- Componente de distribución de tareas
- Componente de equipaje grupal
- Componente de gestión de fechas

Toda la información de un viaje se actualiza en tiempo real para todos los invitados a este, esto es posible gracias a un servidor node.js que conecta la aplicación con una base de datos MySQL. En el servidor se reciben eventos disparados por la aplicación, según el evento y la información que lo acompaña se lee, borra, escribe o actualiza la base de datos, se procesa la información y se envía a la aplicación para actualizar su pantalla.

Abstract

This Final Project involves the development of an application that allows users to plan and manage their trips in a simple way. The operation of this application is as follows. After logging in, users are met with a screen displaying several of their upcoming and past trips and a calendar with all the dates. From this screen, the user can navigate to the specific screen for each trip.

On the trip information screen, the name and planning status of the trip are displayed, along with a list of cards with relevant information about the trip (component cards), and two buttons: one to modify trip information and invite or remove participants, and another to add new components.

The application has 7 different types of components, each for a different aspect of trip planning. Each type of component has two visual representations, the first one being the component card that serves as a preview allowing the user to see a summary of its content and which, if pressed, takes the user to the second representation of the component, a screen that displays all the information and allows the user to update it in real time.

The 7 types of components that can be added to a trip are:

- Room allocation component
- Attendance confirmation component
- Expense management component
- Shopping list component
- Task distribution component
- Group luggage component
- Date management component

All the information about a trip is updated in real time for all the guests invited to it, made possible thanks to a Node.js server that connects the application with a MySQL database. The server receives events triggered by the application, depending on the event and the accompanying information it reads, deletes, writes, or updates the database, processes the information, and sends it back to the application to update its screen.

Tabla de contenidos

1	Introducción.....	1
1.1	Motivación y necesidad del proyecto	1
1.2	Objetivos	2
1.3	Planificación.....	2
1.4	Estructura de la memoria.....	3
2	Soluciones existentes	4
2.1	Herramientas específicamente diseñadas para planificar viajes:	4
2.1.1	TripIt.....	4
2.1.2	Wanderlog.....	4
2.1.3	SaveTrip.....	5
2.2	Herramientas no diseñadas específicamente para planificar viajes, pero útiles en este contexto:	6
2.2.1	WhatsApp	6
2.2.2	Google Drive.....	6
2.2.3	Trello	7
2.2.4	Tricount y Splitwise	8
3	Tecnologías empleadas	9
3.1	Figma.....	9
3.2	Dart	9
3.3	Flutter.....	10
3.4	Otras tecnologías.....	11
3.4.1	Node.js.....	11
3.4.2	MySQL.....	11
3.4.3	Socket.IO	11
4	Diseño	12
4.1	Descripción de la aplicación	12
4.2	Requisitos funcionales.....	13
4.2.1	Pantalla de creación de cuenta.....	13
4.2.2	Pantalla de Login	13
4.2.3	Pantalla de inicio	13
4.2.4	Pantalla de viaje.....	13
4.2.5	Pantalla de Habitaciones	14
4.2.6	Pantalla de confirmación de asistencia.....	14
4.2.7	Pantalla de lista de la compra	15
4.2.8	Pantalla de gestión de deudas	15
4.2.9	Pantalla de tareas	16
4.2.10	Pantalla de equipaje grupal.....	16
4.2.11	Pantalla de Calendario	16
4.3	Requisitos no funcionales.....	17

5	Desarrollo	18
5.1	Base de datos	18
5.2	Servidor.....	22
5.2.1	Inicio de la conexión con la aplicación.....	22
5.2.2	Manejo de eventos.....	22
5.3	Aplicación	26
5.3.1	Información que se conserva en local	26
5.3.2	Definición de objetos	27
5.3.3	Implementación de las pantallas	27
5.3.4	Planteamiento de las pantallas.....	29
6	Evaluación y futuros cambios	39
6.1	Pruebas con usuarios.....	39
6.1.1	Resultados de las pruebas	40
6.1.2	Problemas encontrados	41
7	Conclusiones y futuros cambios.....	42
7.1	Conclusiones.....	42
7.2	Futuros cambios	43
8	Análisis del impacto.....	44
9	Bibliografía	45

Índice de figuras

Figura 1.1 Diagrama de Gantt del proyecto	2
Figura 2.1 Capturas de pantalla de la aplicación TripIt	4
Figura 2.2 Capturas de pantalla de la aplicación Wanderlog	5
Figura 2.3 Capturas de pantalla de la aplicación SaveTrip	5
Figura 2.4 Captura de WhatsApp siendo utilizado para gestionar la compra en un viaje.	6
Figura 2.5 Nueva funcionalidad de WhatsApp para hacer encuestas.....	6
Figura 2.6 Captura de WhatsApp siendo utilizado como galería común para un viaje	6
Figura 2.7 Ejemplos de Google Drive siendo usado para planificar viajes	7
Figura 2.8 Interfaz de Trello	7
Figura 2.9 Interfaz de Tricount.....	8
Figura 2.10 Interfaz de Splitwise	8
Figura 3.1 Evolución de la interfaz en las primeras fases del diseño	9
Figura 4.1 Diagrama de flujo de pantallas de la aplicación.....	12
Figura 5.1 Estructura de la base de datos.....	18
Figura 5.2 Botón de inicio antes y después de conectar con el servidor	26
Figura 5.3 Pantalla de Login sin conexión al servidor	29
Figura 5.4 Pantalla de Login con conexión al servidor	29
Figura 5.5 Pantalla de Login mostrando error de formato en el correo.....	29
Figura 5.6 Pantalla de creación de cuenta vacía.....	30
Figura 5.7 Pantalla de creación de cuenta después de rellenar los campos, muestra errores en los datos	30
Figura 5.8 Pantalla de inicio parte superior	31
Figura 5.9 Pantalla de inicio parte inferior	31
Figura 5.10 Pop up para añadir un nuevo viaje.....	31
Figura 5.11 Pop up para elegir fechas para el viaje	31
Figura 5.12 Calendario con viajes solapados.....	31
Figura 5.13 Pop up para seleccionar uno de los viajes solapados	31
Figura 5.14 Pantalla de viaje con componentes vacíos	32
Figura 5.15 Pantalla de viaje después de modificar los componentes	32
Figura 5.16 Pop up para añadir nuevos componentes.....	32
Figura 5.17 Pop up para actualizar informacion de un viaje e invitar usuarios	33
Figura 5.18 Pop up para invitar usuarios al viaje.....	33
Figura 5.19 Desplegable para actualizar el estado del viaje	33
Figura 5.20 Pop up para añadir una habitación.....	33
Figura 5.21 Interfaz del componente	33
Figura 5.22 Pop up para añadirse o borrarse de la cama	33
Figura 5.23 Pantalla de confirmación sin modificar.....	34
Figura 5.24 Pop up para actualizar estado de la confirmación	34
Figura 5.25 Pantalla de confirmación tras ser modificada	34
Figura 5.26 Pop up para añadir una nueva deuda	35
Figura 5.27 Lista de deudas.....	35
Figura 5.28 Balance del usuario y deudas simplificadas	35
Figura 5.29 Pop up para saldar todas las deudas con una persona.....	35
Figura 5.30 Balance en equilibrio	35
Figura 5.31 Lista de deudas después del pago	35
Figura 5.32 Pop up para añadir un ítem grupal	36
Figura 5.33 Pop up para asignar ítems y actualizar la cantidad necesaria	36
Figura 5.34 Lista de ítems grupales con resumen de lo encargado al usuario	36

Figura 5.35 Pop up para añadir tareas.....	37
Figura 5.36 Pop up para actualizar el estado de una tarea.....	37
Figura 5.37 Listas de tareas.....	37
Figura 5.38 Pop up para añadir producto a la lista	37
Figura 5.39 Ítem marcándose como comprado	37
Figura 5.40 Ítem siendo borrado	37
Figura 5.41 Lista de la compra.....	37
Figura 5.42 Pantalla de fechas con fechas finales y eventos	38
Figura 5.43 Parte inferior de la pantalla.....	38
Figura 5.44 Pantalla de fechas con los eventos de un usuario ocultos.....	38

1 Introducción

En este capítulo se expone la motivación que inspiró la idea para este TFG, los objetivos a cumplir, la planificación y la estructura de la memoria.

1.1 Motivación y necesidad del proyecto

La coordinación de viajes en grupo puede ser todo un desafío que requiere de planificación, trabajo en equipo y organización. Al ser una actividad grupal es necesario que todos los miembros del grupo tengan oportunidad de formar parte del proceso de toma de decisiones pero a medida que aumenta el número de invitados al viaje, se vuelve más difícil encontrar un momento y lugar en el que todos los invitados puedan reunirse, lo que lleva a buscar apoyo en las herramientas TIC.

La tecnología elimina la necesidad de coordinarse tanto en el espacio como en el tiempo, ya que permiten una comunicación asíncrona y a distancia, sin embargo, su potencial en la planificación de viajes en grupo va más allá. Una aplicación diseñada específicamente para la planificación y gestión de viajes no solo puede eliminar la necesidad de reuniones físicas para la toma de decisiones, sino que puede convertirse en una alternativa superior o un complemento valioso a los métodos de planificación más convencionales.

A pesar de este potencial, actualmente existe una brecha en el mercado para este tipo de aplicaciones. Los usuarios terminan recurriendo a herramientas conocidas como WhatsApp o Google Drive, que, aunque útiles, no fueron creadas con este propósito específico. En consecuencia, su utilidad se limita principalmente a la facilitación de la coordinación grupal, sin aportar muchas ventajas adicionales.

Es cierto que existen varias herramientas TIC orientadas a la planificación de viajes pero su enfoque suele ser limitado, encontramos aplicaciones que se centran en mostrar decisiones ya tomadas, otras diseñadas únicamente para planificar la ruta hacia el destino final e incluso aplicaciones que no permiten compartir la información en grupo por lo que carecen de utilidad en este contexto. Además estas aplicaciones dejan de lado otros aspectos claves de la planificación de viaje como el equipaje, la coordinación de fechas, la distribución de las habitaciones del destino y en especial cualquier aspecto que tome importancia una vez el viaje empieza, como la gestión de tareas, los gastos compartidos o la comida entre otras.

La motivación tras este TFG ha sido mi experiencia personal planificando viajes en grupos grandes, la falta de herramientas adecuadas nos llevó a usar los ya mencionados WhatsApp y Google drive, el resultado fue tener que pasar por cientos de mensajes para recordar las decisiones tomadas, confusiones antes y durante el viaje que podrían haberse evitado fácilmente, problemas en la comunicación y en general una experiencia de usuario poco recomendable.

El propósito de este TFG es desarrollar una aplicación que ayude a los usuarios a planificar y gestionar sus viajes en grupo acompañándolos desde las primeras fases de la planificación hasta el fin del viaje. Esta aplicación permitirá a los usuarios acceder a la información relevante más actualizada en todo momento de forma sencilla, representada visualmente para facilitar su comprensión. y actuara como canal de comunicación permitiendo a los usuarios modificar dicha información de forma que los cambios se vean reflejados para el resto de los participantes en tiempo real.

1.2 Objetivos

El objetivo principal del trabajo será el desarrollo de una aplicación multiplataforma completamente funcional que ayude a sus usuarios a gestionar sus viajes, acompañándolos desde su planificación hasta el post viaje.

Este objetivo puede ser dividido en 5 objetivos específicos:

- Familiarización con las tecnologías a emplear.
- Desarrollar una base de datos relacional para persistir los datos y gestionar la autenticación de usuarios.
- Desarrollar el sistema principal de la aplicación.
- Desarrollar un servidor que conecte la base de datos con la aplicación.
- Desarrollar los distintos componentes que los usuarios podrán añadir a su viaje si así lo desean.

La aplicación se basa en un enfoque modular, para cada viaje podrán añadirse componentes como calendarios en los que ver la disponibilidad de cada persona, una sección de pagos en los que apuntar quién paga qué para cuadrar las cuentas, votaciones para elegir los destinos del viaje o votaciones para tomar decisiones sobre el viaje.

1.3 Planificación

Para llevar a cabo este TFG cuento con una lista de las 8 tareas que deben realizarse:

1. Elección de las tecnologías más adecuadas para el proyecto
2. Estudio del estado del técnico de la cuestión
3. Familiarización con las tecnologías a emplear
4. Desarrollo del sistema principal
5. Desarrollo de diferentes módulos opcionales para los usuarios
6. Pruebas con usuarios
7. Desarrollo de la memoria
8. Preparación de la presentación

Cuento también con un plan que establece los tiempos para desarrollar cada una de las tareas previamente mencionadas.



Figura 1.1 Diagrama de Gantt del proyecto

1.4 Estructura de la memoria

En este apartado se muestra un resumen de la estructura de la memoria.

2. Soluciones existentes
Se explican las herramientas actualmente utilizadas en el ámbito de la planificación y gestión de viajes, incluyendo aquellas que fueron específicamente diseñadas con ese fin y aquellas que se desarrollaron con otro objetivo pero que pueden ser usadas para ello.
3. Tecnologías utilizadas
Se explican las herramientas utilizadas para el desarrollo del proyecto, su utilidad y las ventajas que ofrecen.
4. Diseño
Una descripción de la aplicación junto al diagrama de flujo de sus pantallas, seguida de la lista de requisitos usada para guiar la fase de desarrollo.
5. Desarrollo
Se explica en profundidad las 3 partes necesarias para el correcto funcionamiento del proyecto. Primero la base de datos, su estructura y sus tablas, después el servidor, explicando cómo se comunica con la aplicación y la base de datos y como trata los eventos que recibe de la primera, y por último la aplicación, explicando en profundidad el sistema de actualización de la información en pantalla y una descripción del funcionamiento de la interfaz.
6. Evaluación
Una tabla con los resultados de las pruebas con los usuarios junto a una evaluación del sistema, errores detectados durante las pruebas y posibles mejoras que implementar en el futuro.
7. Conclusiones y futuros cambios
Explicación de las conclusiones relacionando cada una con los objetivos expuestos durante la introducción.
8. Análisis de impacto
Explicación del impacto que este trabajo ha provocado a nivel personal así como su potencial impacto a nivel global relacionándolo con los Objetivos de Desarrollo Sostenible (ODS) de las Naciones Unidas.

2 Soluciones existentes

Para analizar el estado del arte en el ámbito de aplicaciones y herramientas para planificar viajes, es importante hacer una separación entre las herramientas específicamente diseñadas para este propósito y aquellas que, aunque no fueron diseñadas con ese fin, también pueden ser utilizadas en este contexto.

2.1 Herramientas específicamente diseñadas para planificar viajes:

2.1.1 TripIt

TripIt [1] es una herramienta que funciona a modo de carpeta en la que juntar información relevante para el viaje. Cuando un usuario recibe confirmaciones de reservas de vuelos, hoteles, alquiler de coches o actividades, puede reenviar estos correos electrónicos a una dirección específica de TripIt. La aplicación extrae y organiza la información relevante, creando un itinerario maestro accesible desde cualquier dispositivo con una interfaz que permite visualizar la información de forma intuitiva como podemos ver en la Figura 1.1.

A diferencia de la aplicación de este TFG, TripIt busca juntar información relevante sobre aspectos del viaje que ya han sido decididos, es decir, no es una herramienta para planear viajes sino más bien una carpeta donde juntar los resultados de la planificación de estos.

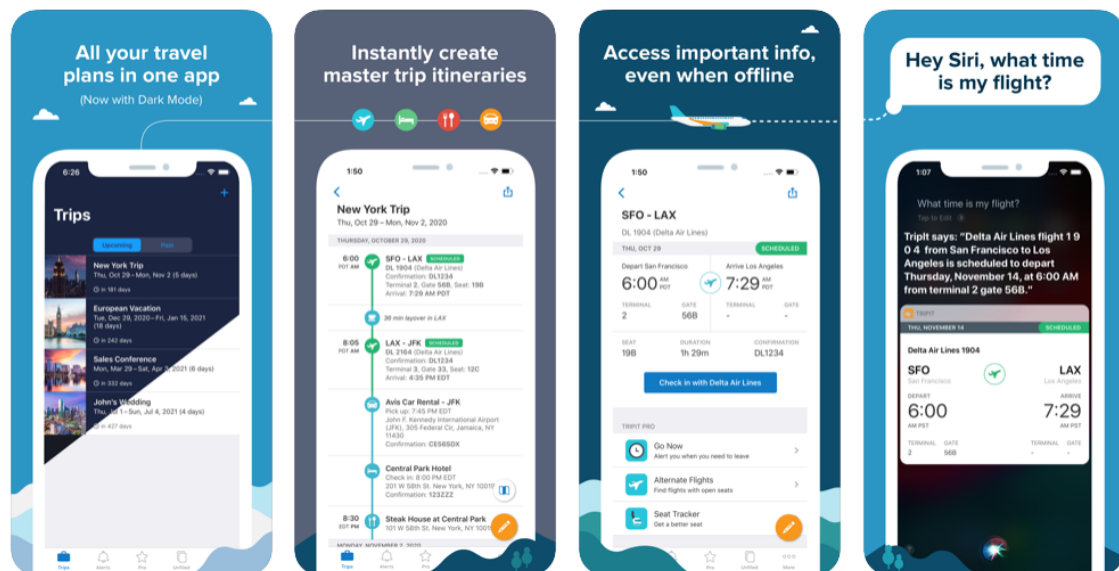


Figura 2.1 Capturas de pantalla de la aplicación TripIt

2.1.2 Wanderlog

Wanderlog [2] (Figura 2.2) es una aplicación y sitio web de planificación de viajes que permite a los usuarios crear, organizar y compartir itinerarios de viaje. Los usuarios pueden agregar hoteles, actividades, restaurantes y otros lugares de interés a su itinerario y ver todo en un mapa interactivo, todo esto en tiempo real, lo que facilita la planificación de viajes en grupo.

Esta es la aplicación más similar a este TFG, la diferencia clave es que esta aplicación busca ayudar a los usuarios a planear un viaje, pero al margen de los itinerarios, no cuenta con herramientas para ayudar a los usuarios durante el mismo. De la misma forma que en Wanderlog un usuario puede añadir a su viaje notas o planes, pierde la oportunidad de añadir otras funcionalidades como gestiones de pagos durante el viaje o distribución de tareas.

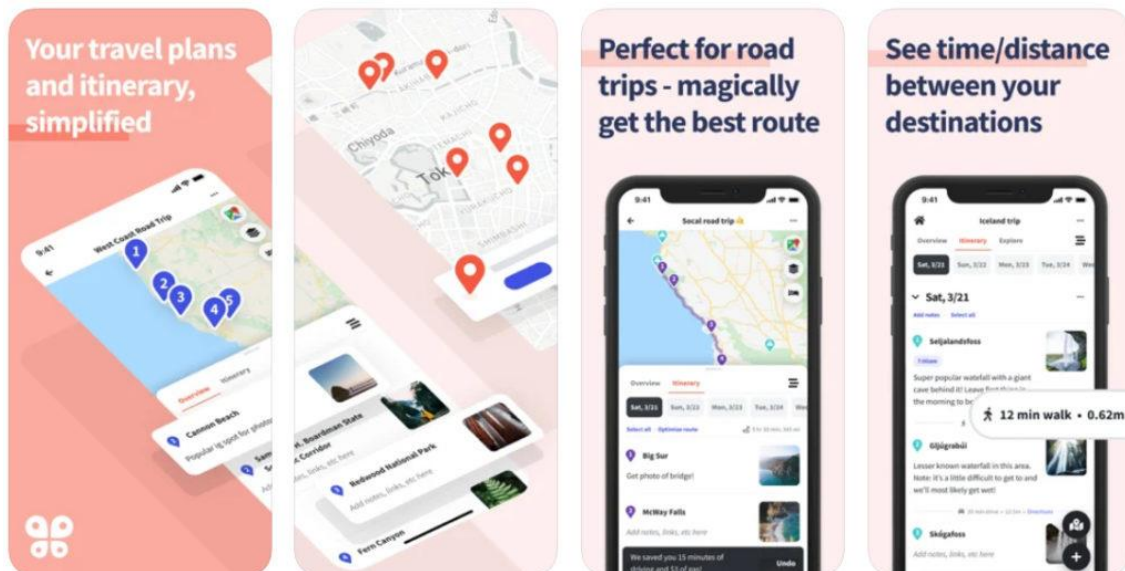


Figura 2.2 Capturas de pantalla de la aplicación Wanderlog

2.1.3 SaveTrip

SaveTrip [3] es muy similar a Wanderlog pero cuenta con varias de las herramientas que los usuarios podrían echar en falta en esta como gestión de gastos y listas de equipaje como se puede ver en la Figura 2.3. Su principal fallo es que SaveTrip no tiene funcionalidad cooperativa por lo que solo es útil para planear viajes en solitario.

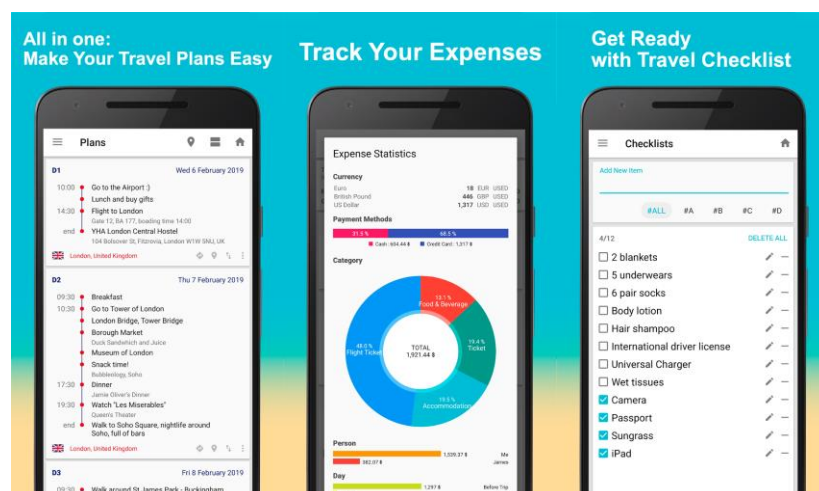


Figura 2.3 Capturas de pantalla de la aplicación SaveTrip

2.2 Herramientas no diseñadas específicamente para planificar viajes, pero útiles en este contexto:

2.2.1 WhatsApp

Aunque es una aplicación de mensajería, WhatsApp [4] puede ser útil para planificar viajes en grupo. Los usuarios pueden crear grupos para discutir detalles del viaje, compartir documentos, ubicaciones, usando el chat a modo de lista fijando mensajes, o a modo de carpeta compartida para guardar fotos. Desde hace poco es posible incluso enviar votaciones por WhatsApp en las que cada usuario puede marcar una o más opciones. Desde la Figura 2.4 a la 2.6 se pueden ver varios casos de uso de WhatsApp para la planificación de viajes.



Figura 2.4 Captura de WhatsApp siendo utilizado para gestionar la compra en un viaje.

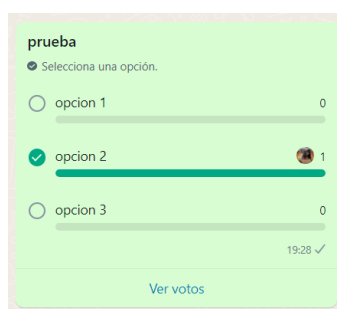


Figura 2.5 Nueva funcionalidad de WhatsApp para hacer encuestas.

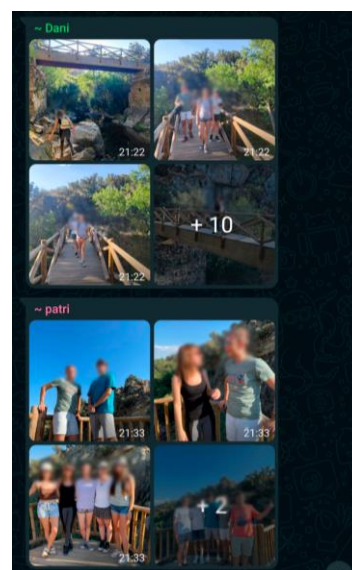


Figura 2.6 Captura de WhatsApp siendo utilizado como galería común para un viaje

2.2.2 Google Drive

Los documentos creados y editados en Google drive [5] se actualizan en tiempo real para los usuarios con acceso a estos lo que convierte Google drive en una plataforma muy útil para la planificación de viajes.

Por ejemplo, Google Docs nos permite tener un documento de texto compartido con varios usuarios por lo que resulta muy útil como de lista, por ejemplo, para juntar todos los enlaces relevantes para un viaje como puedan ser links a los billetes de avión, a las opciones de apartamentos que alquilar o a guías turísticas entre otros y Google Sheets además de su habitual uso como herramienta de contabilidad, puede ser usada también para planificar otros aspectos de los viajes como por ejemplo las fechas libres de cada usuario y aunque no sea la solución más elegante, es una de las opciones más simples que hay. En la Figura 2.7 se pueden ver ejemplos de estas herramientas siendo usadas para planificar el destino y fechas de un viaje.

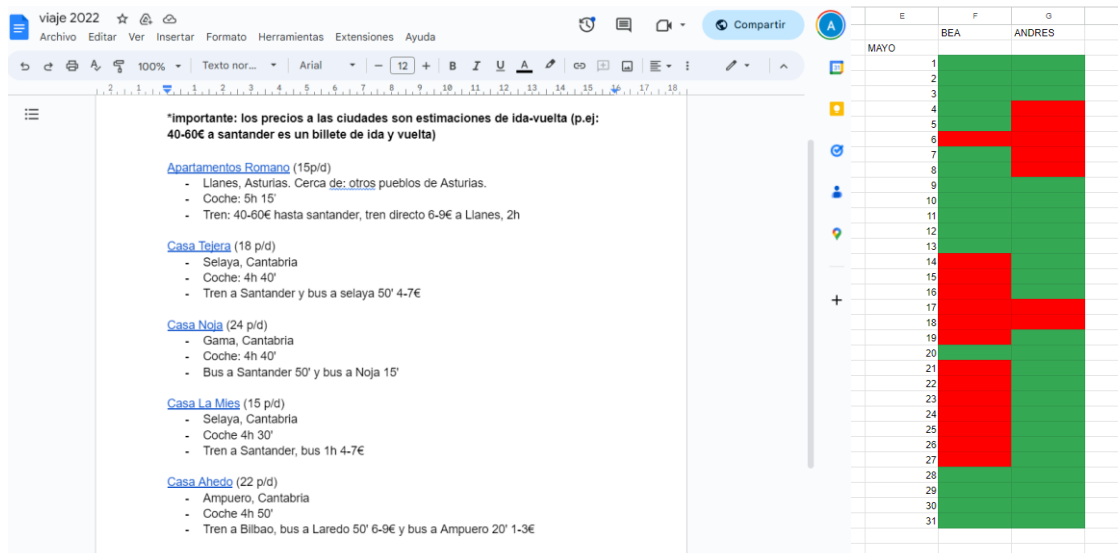


Figura 2.7 Ejemplos de Google Drive siendo usado para planificar viajes

2.2.3 Trello

Trello [6] (Figura 2.8) es una herramienta de gestión de proyectos basada en tableros, está orientada a proyectos empresariales, pero algunas de sus funcionalidades pueden ser útiles para gestionar un viaje en grupo. Trello cuenta con herramientas para gestión de fechas y eventos, además los usuarios pueden crear listas y tarjetas para organizar diferentes aspectos del viaje, como reservas, actividades y gastos. Trello cuenta con una opción gratuita que aun estando bastante limitada puede seguir siendo útil para la gestión de viajes.

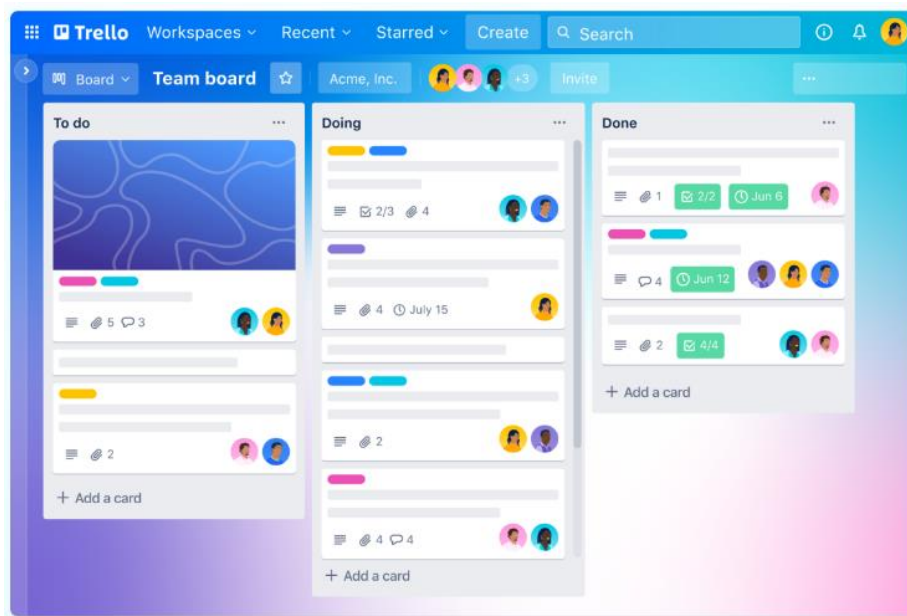


Figura 2.8 Interfaz de Trello

2.2.4 Tricount y Splitwise

Tricount [7] y Splitwise [8] a los usuarios gestionar los pagos en un viaje en grupo, cuando un usuario paga algo que debe ser dividido entre varias personas añade a la aplicación el gasto, quien lo ha pagado y entre que personas debe ser dividido.

La principal ventaja de estas aplicaciones es que, al acabar el viaje, simplifican el proceso de pago ya que tienen un sistema que reduce la cantidad de transacciones necesarias, por ejemplo, si A debe 5 euros a B, B debe 5 euros a C y C debe 6 euros a A, solo sería necesario que C pague 1 euro a A.

Esta simplificación es muy útil en viajes en grupo ya que es común que una sola persona pague por todos y luego se hagan cuentas más adelante pero cuando el número de miembros del viaje aumenta esta tarea se vuelve muy compleja. Además de simplificar el proceso de pagos estas aplicaciones cuentan con una interfaz para mostrar la información a los usuarios de forma intuitiva como puede verse en la Figura 2.9 y la Figura 2.10

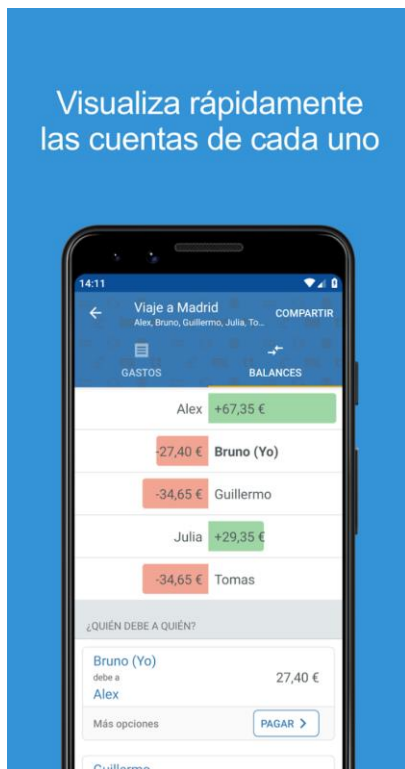


Figura 2.9 Interfaz de Tricount

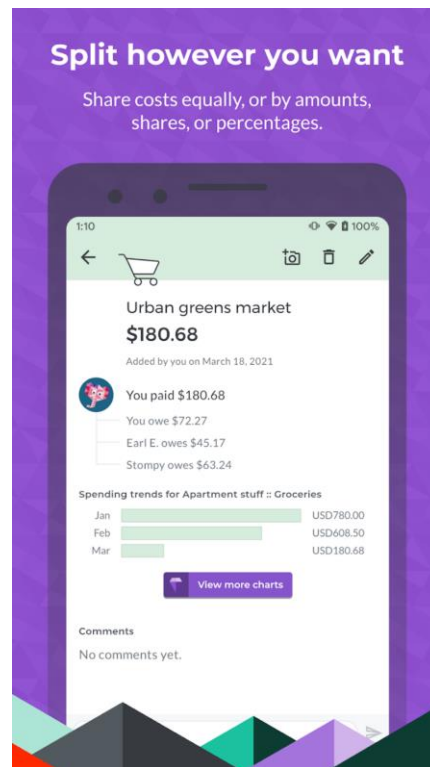


Figura 2.10 Interfaz de Splitwise

3 Tecnologías empleadas

En este capítulo se explican las tecnologías utilizadas para el desarrollo del proyecto, para que se utilicen y que ventajas tienen.

3.1 Figma

Para los primeros diseños de la interfaz de usuario (Figura 3.1) he utilizado Figma [9], una herramienta basada en la nube que permite diseñar interfaces de usuario de forma rápida y sencilla. El uso de componentes predefinidos como tarjetas con altura y sombras me permitió hacer varios diseños en poco tiempo para hacerme una idea de cómo quería que fuera mi aplicación incluso antes de empezarla.

Además, Figma cuenta con soporte multiplataforma y con funcionalidades cloud permitiéndome diseñar la aplicación desde mi ordenador y visualizarla he interactuar con ella al momento en mi móvil.

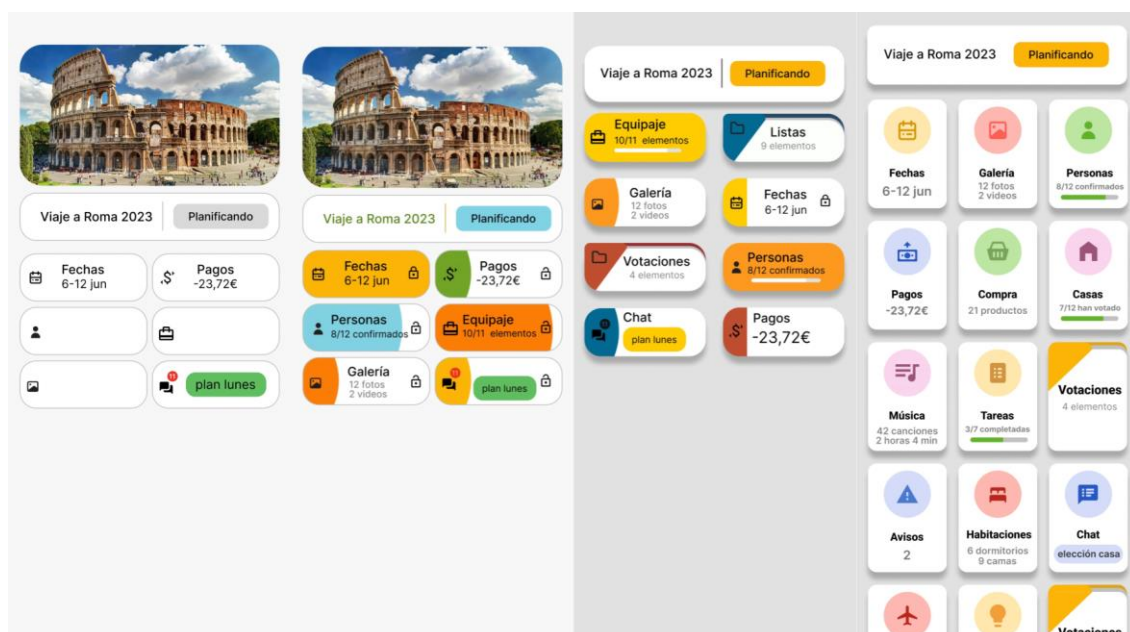


Figura 3.1 Evolución de la interfaz en las primeras fases del diseño

3.2 Dart

La decisión más importante para el desarrollo de este trabajo fue elegir un lenguaje de programación para la aplicación cliente. Tras mucho investigar decidí desarrollar el proyecto utilizando Dart [10], un lenguaje de programación desarrollado por Google que está ganando popularidad en los últimos años. Algunas de sus principales ventajas son:

Orientado a objetos:

Dart es un lenguaje de programación orientado a objetos lo que facilita la creación de estructuras de datos complejas y reutilización de código mediante herencia y polimorfismo.

Compilación Just in Time (JIT):

El código escrito en Dart puede ser compilado a código máquina en tiempo real, es decir, no es necesario detener la ejecución del programa para ver en tiempo real los nuevos cambios.

Null Safety:

La null safety es una característica de Dart que distingue entre variables que pueden ser nulas y las que no pueden serlo, ayudando a prevenir errores de null pointer. Existen tipos distintos para las variables que puedan ser nulas, por ejemplo, un String en Dart debe tener un valor, pero si existe la posibilidad de que no se le asigne ninguno, Dart obliga a cambiar el tipo a “String?” y obliga a manejar explícitamente los casos nulos, lo que resulta en código más seguro y predecible, y reduce la cantidad de errores en las aplicaciones.

Fácil de aprender:

La sintaxis de Dart es sencilla y similar a otros lenguajes de programación orientados a objetos que he utilizado previamente como Java o C#.

Flutter:

Un framework de Dart creado por Google para el desarrollo de aplicaciones móviles multiplataforma que fue la principal la razón para elegir Dart. Más adelante hablare en detalle de esta herramienta.

3.3 Flutter

Flutter es un framework para Dart creado por Google. Se utiliza para desarrollar aplicaciones para Android, iOS, Linux, Mac, Windows, y la web desde una única base de código. Algunas de sus principales ventajas y razones por las que lo he elegido son:

Rendimiento:

Al compilar a código nativo y tener su propio motor de renderizado, Flutter puede proporcionar un rendimiento superior en comparación con otros marcos de desarrollo multiplataforma.

Widgets:

Los componentes que forman la interfaz de la aplicación Flutter se denominan Widgets. Los Widgets se organizan en una estructura de árbol, cada Widget puede tener uno o más Widgets hijo dependiendo del componente. Estos Widgets vienen predefinidos y son personalizables lo que facilitan el diseño de la interfaz y permite crear aplicaciones muy similares a las programadas de forma nativa ya que pueden importarse Widgets con diseño Material Design (El utilizado en los dispositivos Android) y Cupertino (Para dispositivos IOS) y mostrar unos u otros dependiendo de la plataforma en la que se ejecute la aplicación.

Hot Reload:

Flutter utiliza la compilación JIT de Dart para ofrecer la funcionalidad de Hot Reload, que permite ver los cambios en el código en tiempo real sin necesidad de volver a compilar la aplicación y sin perder el estado de las variables. Esta funcionalidad es particularmente útil ya que además de permitir ver la aplicación mientras se desarrolla, lo que facilita el diseño de la interfaz y la resolución de errores ya que el programa no termina su ejecución cuando falla y puede ser arreglado en el momento sin necesidad de recompilarlo.

3.4 Otras tecnologías

3.4.1 Node.js

Node.js [12] es un entorno de ejecución de JavaScript [13] diseñado para construir aplicaciones de red escalables y eficientes, especialmente servidores de entrada/salida orientados a eventos, lo que lo hace ligero y eficiente, perfecto para aplicaciones de datos en tiempo real.

3.4.2 MySQL

MySQL [14] es un sistema de gestión de bases de datos relacional de código abierto. Es conocido por su velocidad y confiabilidad, así como por su escalabilidad. MySQL es utilizado en una amplia variedad de aplicaciones, desde aplicaciones web a gran escala hasta almacenamiento de datos incrustados. En mi caso, lo he utilizado para almacenar y gestionar los datos de mi aplicación, garantizando su accesibilidad y seguridad.

3.4.3 Socket.IO

Socket.IO [15] es una biblioteca de JavaScript que permite la comunicación bidireccional en tiempo real entre el servidor y los clientes web o móviles. Permite que el servidor envíe datos a los clientes cuando tiene nuevos datos disponibles, sin necesidad de que los clientes hagan una solicitud HTTP específica. Este patrón de comunicación es muy útil para las aplicaciones que necesitan actualizaciones en tiempo real.

3.4.3.1 No IP

No-IP [16] es un servicio que permite asignar un nombre de dominio a una dirección IP dinámica. Esto es particularmente útil cuando se aloja un servidor en una ubicación con una dirección IP pública que puede cambiar, como es mi caso. Con No-IP, aunque la dirección IP cambie, los usuarios pueden acceder al servidor utilizando el mismo nombre de dominio.

3.4.3.2 Visual Studio Code

Visual Studio Code [17] es un editor de código fuente desarrollado por Microsoft. Es altamente personalizable. Cuenta con una gran cantidad de plugins que pueden ser instalados para mejorar y adaptar sus funcionalidades al gusto y las necesidades de cada desarrollador.

3.4.3.3 Android Studio

Android Studio [18] es un entorno de desarrollo integrado (IDE) para el desarrollo de aplicaciones para Android. Proporciona un conjunto de herramientas que facilitan la creación de aplicaciones de Android, en mi caso he utilizado su emulador para probar mi aplicación en varios dispositivos Android virtuales y ver en tiempo real mi aplicación sin tener que compilarla de nuevo con cada cambio.

4 Diseño

En este apartado se explicará el diseño de la aplicación de planificación de viajes en grupo. Primero se da una explicación del funcionamiento de la aplicación acompañado por un diagrama de flujo de pantallas y a continuación una lista de los requisitos de esta.

4.1 Descripción de la aplicación

Esta aplicación es una herramienta para la planificación de viajes en grupo. La información del viaje se organiza en componentes que la muestran de forma intuitiva para los usuarios y les permite modificarla interactuando con los elementos de la pantalla de cada componente. El diagrama de flujo de pantallas puede verse en la Figura 4.1.

Al tocar en un viaje el usuario es llevado a una pantalla en la que se muestran los distintos componentes que forman el viaje, así como la opción de añadir componentes nuevos. Estos componentes son pequeñas tarjetas en filas de 3 que muestran un resumen de la información que contienen. Tocar cualquiera de estos componentes nos llevara a una pantalla que será distinta según el tipo de componente, en esta pantalla veremos la información completa de dicho componente y podremos editarla según sea necesario. Por ejemplo, un componente de confirmaciones mostrara información relativa a la asistencia o no asistencia de los miembros invitados al viaje y un componente de tipo habitaciones mostrara información relativa a las habitaciones y camas que ocuparan los usuarios.

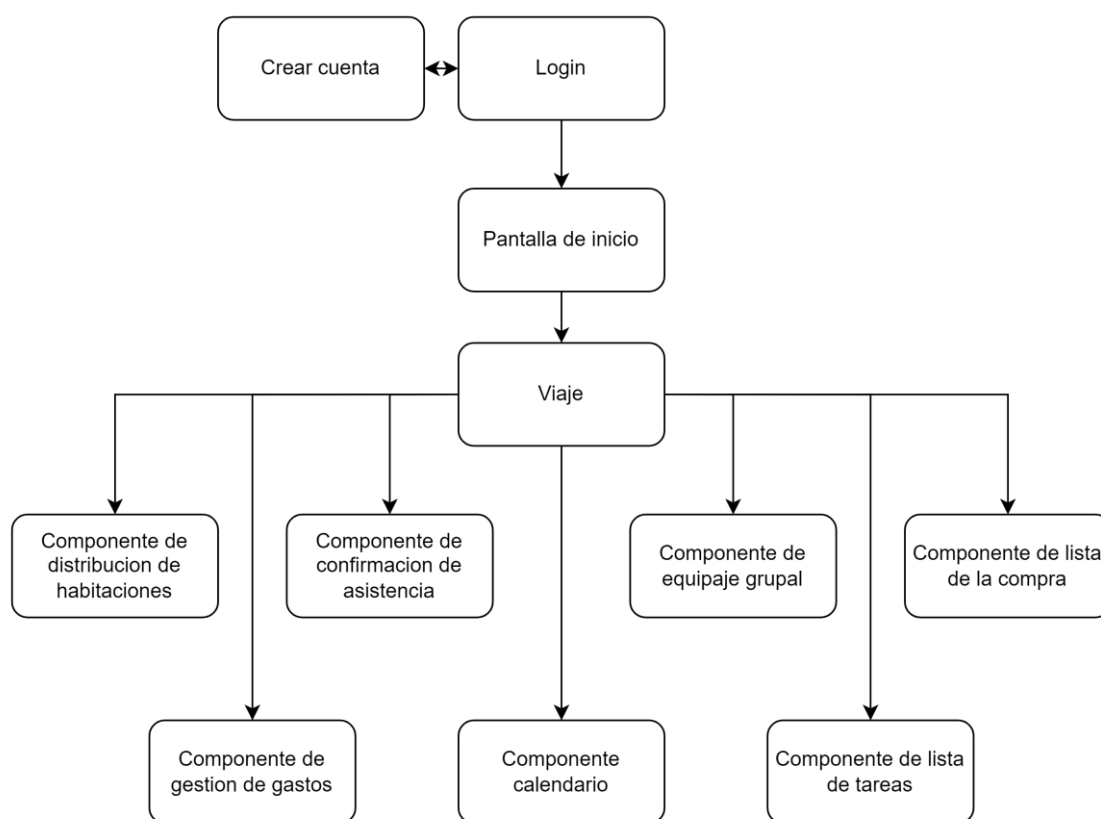


Figura 4.1 Diagrama de flujo de pantallas de la aplicación

4.2 Requisitos funcionales

4.2.1 Pantalla de creación de cuenta

R01: Registro de usuarios: Los usuarios pueden registrarse en la aplicación proporcionando un nombre, primer y segundo apellido, correo electrónico, contraseña y un color para el perfil en la pantalla de creación de cuenta.

4.2.2 Pantalla de Login

R02: Inicio de sesión: Los usuarios pueden iniciar sesión proporcionando su correo y su contraseña en la pantalla de Login.

4.2.3 Pantalla de inicio

R03: Próximo viaje: La aplicación muestra en la pantalla de inicio información sobre el viaje más cercano incluyendo su foto, su nombre y los días restantes para que empiece.

R04: Calendario de viajes: La aplicación muestra en la pantalla de inicio un calendario que muestre las fechas de todos los viajes del usuario, marcando en rojo las fechas en las que dos o más viajes se solapan.

R05: Navegación a viajes: Al tocar una fecha que pertenezca a un viaje en el calendario de la pantalla de inicio, la aplicación lleva al usuario a la pantalla de dicho viaje.

R06: Gestión de viajes solapados: Al tocar una fecha que pertenezca a más de un viaje en el calendario de la pantalla de inicio, la aplicación debe mostrar un Pop Up con la lista de los viajes que ocupan ese día.

R07: Navegación a viajes solapados: Al tocar uno de los viajes mostrados al seleccionar un día con varios viajes en el calendario de la pantalla de inicio, debe llevar al usuario a la pantalla de dicho viaje.

R08: Visualización de próximos viajes: La aplicación muestra en la pantalla de inicio una lista con los próximos viajes del usuario, con su nombre, estado y foto, tocar la foto del viaje lleva al usuario a la pantalla de dicho viaje.

R09: Visualización de viajes pasados: La aplicación muestra en la pantalla de inicio una lista con los próximos viajes del usuario, con su nombre, estado y foto, tocar la foto del viaje lleva al usuario a la pantalla de dicho viaje.

R10: Creación de viajes: Los usuarios pueden crear un nuevo viaje desde la pantalla de inicio proporcionando un nombre, un URL para la imagen, una fecha de inicio y una fecha de final.

4.2.4 Pantalla de viaje

R11: Visualización de detalles de viajes: En la parte superior de la pantalla de un viaje los usuarios pueden ver el nombre y el estado de este.

R12: Modificación de un viaje Los usuarios pueden modificar las siguientes propiedades de un viaje.

R12.1: Nombre del viaje

R12.2: Invitados al viaje

R12.3: Estado del viaje

R12.4: Fecha de inicio del viaje

R12.5: Fecha de fin del viaje

R13: Borrado de un viaje: Los usuarios pueden borrar los viajes a los que tienen acceso. Borrar un viaje implica además borrar todos los componentes que contenga.

R14: Visualización de componentes: La aplicación muestra en la pantalla de un viaje todos sus componentes mostrando para cada uno un resumen de la información relevante del componente.

R15: Creación de componentes: Los usuarios pueden crear un nuevo componente dando un nombre y eligiendo de una lista el tipo de componente y el color de su icono. En el caso del componente de confirmación de asistencia se crean dentro del componente una confirmación con estado pendiente por cada usuario invitado al viaje en el momento de la creación del componente.

R16: Navegación a pantallas de componentes: Al tocar sobre un componente de la lista el usuario es movido a la pantalla de dicho componente.

4.2.5 Pantalla de Habitaciones

R17: Borrado del componente de Habitaciones: Los usuarios pueden borrar el componente de Habitaciones de un viaje. Borrar un componente de distribución de habitaciones borra todas las habitaciones que contenga.

R18: Visualización de Habitaciones: Los usuarios pueden ver todas las habitaciones y camas existentes en el componente de Habitaciones, incluyendo las asignaciones de usuarios a las camas.

R19: Creación de Habitaciones: Los usuarios pueden añadir nuevas habitaciones al componente de Habitaciones proporcionando un nombre para la nueva habitación.

R20: Borrado de Habitaciones: Los usuarios pueden borrar habitaciones existentes del componente de Habitaciones. Borrar una habitación borra todas las camas que contenga.

R21: Creación de Camas: Dentro de una habitación existente, los usuarios pueden crear nuevas camas proporcionando un nombre para la nueva cama.

R22: Borrado de Camas: Los usuarios pueden borrar camas existentes de una habitación. Borrar una cama borra todas las asignaciones que contenga.

R23: Asignación de Camas: Los usuarios pueden asignarse a sí mismos a una cama existente en una habitación.

R24: Borrado de asignación de Camas: Los usuarios pueden desasignarse a sí mismos de una cama a la que estén asignados.

4.2.6 Pantalla de confirmación de asistencia

R25: Borrado del componente de Habitaciones: Los usuarios pueden borrar el componente de Habitaciones de un viaje. Borrar un componente de confirmación de asistencia borra todas las confirmaciones que contiene.

R26: Visualización de Confirmaciones: Los usuarios pueden ver tres listas de confirmaciones, cada una muestra los usuarios cuyo estado de confirmación corresponde con el título de la lista que ocupa. Estas listas son "Confirmado", "Pendiente" y "No asistirá".

R27: Actualización de confirmaciones Los usuarios pueden actualizar el estado de su propia confirmación seleccionando el nuevo estado en una lista con sus tres valores posibles.

4.2.7 Pantalla de lista de la compra

R28: Borrado del componente de Lista de la Compra: Los usuarios pueden borrar el componente de Lista de la Compra de un viaje. Borrar un componente de Lista de la Compra borra todos los productos que contiene.

R29: Visualización de productos pendientes de compra: Los usuarios pueden ver una lista de todos los productos pendientes de ser comprados.

R30: Visualización de productos comprados: Los usuarios pueden ver una lista de todos los productos que ya han sido comprados.

R31: Actualización del estado de los productos: Los usuarios pueden marcar un producto de la lista de pendientes como comprado, lo que provoca que se mueva a la segunda lista.

R32: Eliminación de productos: Los usuarios pueden eliminar producto de la lista de pendientes. Una vez comprado no se pueden eliminar.

4.2.8 Pantalla de gestión de deudas

R33: Borrado del componente de Deudas: Los usuarios pueden borrar el componente de Deudas de un viaje. Borrar un componente de Deudas borra todas las deudas que contiene.

R34: Visualización de Deudas: Los usuarios pueden ver todas las deudas del viaje, mostrando para cada deuda el acreedor, los deudores, la cantidad total prestada por el acreedor y en caso de que el usuario esté involucrado en la deuda como deudor o acreedor, la cantidad de dinero que debe recibir o pagar por esa deuda.

R35: Creación de Deudas: Los usuarios pueden crear proporcionando un nombre para la deuda, la cantidad prestada, seleccionando de una lista al usuario acreedor y marcando en una lista a todos los usuarios deudores.

R36: Simplificación de deudas: La aplicación simplifica las deudas del componente para que los usuarios tengan que realizar la mayor cantidad de transferencias posibles dando como resultado el balance total de cada usuario y una lista de deudas simplificadas en las que solo hay un deudor y un acreedor.

R37: Visualización de balance: Los usuarios pueden ver su balance actual para el componente de pago en el que se encuentran, este indica la cantidad de dinero que el usuario debe al resto o el dinero que le deben a él.

R38: Visualización de Deudas simplificadas: Los usuarios pueden ver una lista con las deudas simplificadas de cada uno. Las deudas en las que el usuario que está usando la aplicación sea el deudor tienen al lado un botón que permite crear una deuda con el nombre “pago deudas” que cancele la deuda simplificada ya que se crea con la misma cantidad que esta, pero con los papeles de deudor y acreedor intercambiados. Esto se hace para indicar que una deuda ha sido pagada.

4.2.9 Pantalla de tareas

R39: Borrado del componente de Tareas: Los usuarios pueden borrar el componente de Tareas de un viaje. Borrar un componente de Tareas borra todas las tareas que contiene.

R40: Creación de Tareas: Los usuarios pueden crear nuevas tareas proporcionando un nombre para la tarea y asignando usuarios seleccionándolos de una lista con los invitados al viaje.

R41: Visualización de Tareas: Los usuarios pueden ver todas las tareas existentes en el componente de Tareas, divididas en dos listas: una lista de tareas pendientes y una lista de tareas terminadas.

R42: Actualización del estado de las Tareas: Los usuarios pueden cambiar el estado de una tarea de pendiente a terminada y viceversa.

R43: Borrado de Tareas: Los usuarios pueden borrar tareas existentes del componente de Tareas. Borrar una tarea borra todas las asignaciones de encargados que contenga.

4.2.10 Pantalla de equipaje grupal

R44: Borrado del componente de Equipaje Grupal: Los usuarios pueden borrar el componente de Equipaje Grupal de un viaje. Borrar un componente de Equipaje Grupal borra todos los ítems que contiene.

R45: Creación de Ítems: Los usuarios pueden crear nuevos ítems proporcionando un nombre para el ítem y la cantidad necesaria. Crear un ítem crea automáticamente una asignación de dicho ítem para cada usuario invitado al viaje con cantidad inicial 0.

R46: Borrado de Ítems: Los usuarios pueden borrar ítems existentes del componente de Equipaje Grupal. Borrar un ítem borra todas sus asignaciones.

R47: Visualización de Ítems: Los usuarios pueden ver todos los ítems existentes en el componente de Equipaje Grupal, incluyendo las asignaciones de cada ítem.

R50: Modificación de asignaciones: Los usuarios pueden modificar la cantidad asignada de un ítem a cualquier usuario. El objetivo es que la suma de las cantidades de las asignaciones sea igual o superior a la cantidad necesaria del ítem.

R51: Modificación de la cantidad necesaria de un Ítem: Los usuarios pueden modificar la cantidad necesaria de cualquier ítem.

4.2.11 Pantalla de Calendario

R52: Borrado del componente de Calendario: Los usuarios pueden borrar el componente de Calendario de un viaje. Borrar un componente de Calendario borra la fecha seleccionada y todos los eventos de los usuarios.

R53: Visualización del Calendario: Los usuarios pueden ver un calendario que refleja un rango de fechas de color verde que representa la fecha actualmente elegida como resultado de la planificación, días marcados amarillo y días marcados en rojo. Estos días amarillos y rojos son llamados eventos y representan días en los que alguno de los invitados al viaje preferiría que no coincidan con el viaje (amarillo) o directamente no podría asistir si coincide (rojo).

R54: Modificación de la fecha elegida: Los usuarios pueden modificar la fecha elegida en el calendario.

R55: Gestión de eventos: Los usuarios pueden añadir y eliminar sus propios eventos.

R56: Filtrado de eventos: Los usuarios pueden desmarcar de una lista con todos los invitados a los usuarios cuyos eventos no quieren ver en el calendario para facilitar la planificación de la fecha final.

4.3 Requisitos no funcionales

R57: Conexión con el servidor: La aplicación debe mantener una conexión constante con el servidor. En caso de que se pierda la conexión, la aplicación debe intentar reconectarse automáticamente hasta que se restablezca la conexión.

R58: Actualización de datos: El sistema debe mostrar en todo momento la información más actualizada posible. Para lograrlo cada acción de un usuario que afecte a la información guardada en la base de datos implica que se actualice la información de todos los clientes afectados por el cambio inmediatamente.

R59: Seguridad de la base de datos: Los accesos a la base de datos se realizan con un usuario que no tiene privilegios root, para limitar el potencial daño en caso de un ataque de seguridad.

R61: Cifrado de contraseñas: Las contraseñas de los usuarios se almacenan de forma segura mediante un proceso de hashing. Esto significa que incluso si los datos de la base de datos son comprometidos, las contraseñas de los usuarios no estarán expuestas.

R62: Consistencia: La interfaz de usuario de la aplicación debe ser consistente en todas las pantallas y flujos de trabajo. Esto ayuda a los usuarios a entender cómo funciona la aplicación y a predecir cómo se comportará.

R63: Rendimiento: La aplicación debe responder rápidamente a las interacciones del usuario para proporcionar una experiencia de usuario fluida y agradable.

R64: Diseño intuitivo: La aplicación debe ser fácil de usar y entender para los usuarios. Los usuarios deben ser capaces de realizar tareas comunes de forma rápida y eficiente sin necesidad de formación previa.

5 Desarrollo

En este apartado de la memoria se describe el trabajo realizado en el TFG. Empezando por la base de datos mostrando su estructura y explicando la función y las decisiones tomadas en cada una de sus tablas. A continuación se explicará el funcionamiento del servidor que conecta la base de datos con la aplicación y por último una explicación en detalle del funcionamiento de la aplicación junto a una descripción de todas sus pantallas.

5.1 Base de datos

La base de datos de este proyecto cumple dos funciones. La primera y más evidente función es la persistencia de datos, ya que almacena toda la información relevante para los usuarios de forma que si estos cierran la aplicación o incluso pierden su dispositivo, aún pueden acceder a esta sin problemas. Además, la base de datos actúa como un almacenamiento compartido, al dar acceso a varios usuarios a un conjunto determinado de información permitiéndoles añadir, eliminar, borrar y actualizar su contenido, se crea un canal de comunicación entre los usuarios. La estructura de la base de datos puede verse en la Figura 5.1.

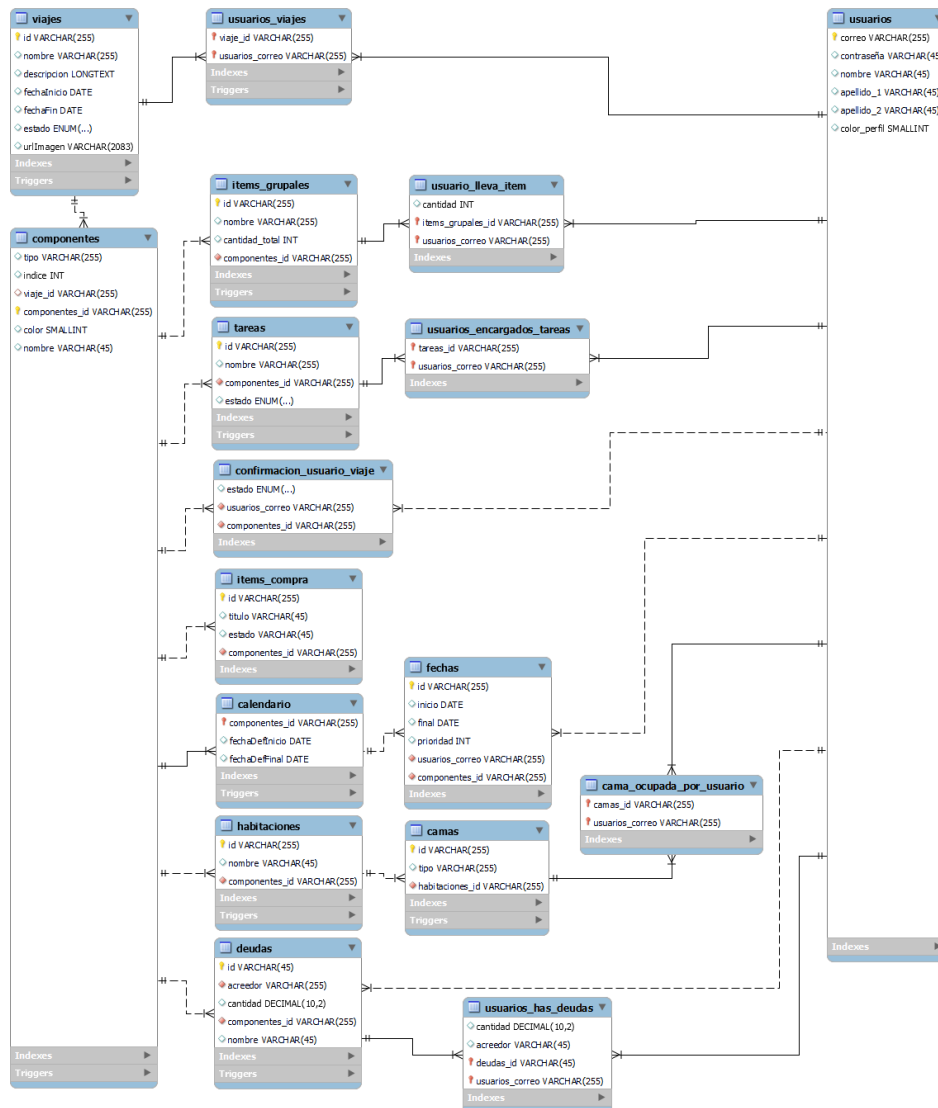


Figura 5.1 Estructura de la base de datos

Usuarios:

Los usuarios se crean en la base de datos con un identificador que es su correo, una contraseña dos apellidos y color del perfil ya que en algunos componentes se muestra una miniatura de los usuarios usando la primera letra de su nombre y la primera letra de su apellido sobre un fondo del color que hayan elegido al crear su cuenta.

Viajes:

Los viajes tienen un identificador autogenerado, un nombre, una descripción que actualmente se crea vacía porque aún no se usa, una fecha de inicio, una fecha de final, un URL para la imagen del viaje y un estado que puede tomar los valores 'Planificando', 'Planificado', 'En curso', 'Finalizado' y 'Cancelado'

Componentes:

Los componentes están diseñados de forma que implementan la llamada estructura de los modelos Entidad relación extendidos. Todos los componentes cuentan con un identificador autogenerado, un tipo, un índice para su posición en la lista de componentes de la aplicación, un id del viaje al que pertenece el componente que es clave foránea de la tabla viajes, un int para el color de forma similar a el color del usuario, y un nombre.

En la aplicación y el servidor, además de estas propiedades todos los objetos componente tienen otra llamada subcomponente que consiste en una lista de objetos con propiedades distintas según el tipo del componente. Por ejemplo, un componente de tipo Tareas tendrá en su subcomponente una lista de objetos tarea, cada una con su nombre, su estado, su id autogenerado y la lista de usuarios encargados. Por otro lado, un componente de tipo distribución de habitaciones tendrá en su subcomponente una lista de objetos habitación, cada habitación de la lista tiene sus propiedades y una lista con las camas que contiene, cada cama de nuevo con sus propiedades y una lista de los usuarios que la ocupan.

Las filas de las 7 tablas que están conectadas a la tabla componentes sin contar la tabla de viajes y que almacenan la información necesaria para la propiedad subcomponente, reciben como Foreign de la tabla componentes el id del componente al que pertenecen. En una misma tabla puede haber varias filas con el mismo valor en esta columna, por ejemplo, puede haber varias tareas dentro de un mismo componente, pero nunca puede ocurrir que dos tablas distintas comportan este valor, por ejemplo, es imposible que un mismo componente contenga tareas y deudas.

Esta forma de diseñar la base de datos para que distintos componentes puedan tener distintas propiedades se conoce como modelo Entidad-Relación Extendido (EER). Este modelo permite una mayor flexibilidad y adaptabilidad en el diseño de la base de datos, permitiendo que los componentes puedan tener propiedades distintas según su tipo. Esto facilita la gestión y manipulación de los datos, y permite una representación más precisa y detallada de la información en la base de datos.

Confirmaciones

La tabla confirmaciones tiene en cada fila el id de un usuario, tomado como clave foránea de la tabla 'usuarios', el id del componente correspondiente, también como clave foránea, y un estado, que es un campo enumerado que puede tener los valores "pendiente", "confirmado" o "no asistirá".

Cuando se crea un nuevo componente de asistencia en un viaje, este genera automáticamente una fila en la tabla 'confirmación_usuario_viaje' para cada usuario invitado al viaje. Esta fila contiene el id del usuario, el id del componente y el estado inicial "pendiente".

Además, si un usuario se añade a un viaje posteriormente, un disparador (trigger) se encargará de insertar la nueva fila correspondiente a dicho usuario en la tabla. De igual manera, si se elimina un componente de asistencia, otro disparador se activará para eliminar todas las filas asociadas a ese componente en la tabla.

Calendario

La tabla calendario tiene 3 propiedades: la primera es el id del componente al que pertenece que además de ser una clave foránea es la clave primaria de la fila, las dos propiedades restantes son una fecha de inicio y una fecha de finalización, ya que el objetivo del componente 'calendario' es coordinar una fecha común entre todos los invitados al viaje.

Fechas

Además, el componente 'calendario' puede tener asociados varios eventos. Cada usuario puede marcar en el calendario común los días en los que no está disponible o los días en los que preferiría no programar el viaje. Al hacerlo se añade a la tabla fechas una nueva fila, esta fila tendrá 6 propiedades que son: Un identificador autogenerado, una fecha de inicio, una fecha fin, el id del componente al que pertenece y el id del usuario que ha añadido el evento.

Items grupales

Cada ítem grupal tiene 4 propiedades: un identificador único, un nombre, la cantidad de unidades de dicho ítem que se consideran necesarias y el identificador del componente al que pertenece que se obtiene como clave foránea de la tabla componentes.

Usuario lleva ítem

Al crear una nueva fila en la tabla items_grupales se activa un trigger que crea una nueva fila en la tabla usuario_lleva_item por cada usuario invitado al viaje al que pertenece dicho ítem. Las filas de esta tabla representan asignaciones de ítems a usuarios. La tabla tiene 3 propiedades: la primera es la cantidad del ítem que el usuario tiene asignada, se crea con valor 0, la segunda propiedad es el identificador del ítem en cuestión que se obtiene como clave foránea de la tabla items_grupales, y la tercera propiedad es el identificador del usuario que se obtiene como clave foránea de la tabla usuarios.

Deudas

La tabla deudas tiene 5 propiedades: un identificador, una cantidad que representa el dinero pagado por el acreedor de la deuda, el nombre de la deuda, el identificador del usuario acreedor que se obtiene como clave foránea de la tabla usuarios y el identificador del componente al que pertenece la deuda que se obtiene como clave foránea de la tabla componentes.

Usuarios_has_deudas

Una deuda puede tener varios deudores, al crear una deuda además del acreedor deben incluirse los usuarios deudores que se reparten el dinero a pagar, cada una de estas deudas que pertenecen a la deuda padre se almacenan como filas en la tabla usuario_has_deudas con 4 propiedades: la cantidad a pagar por cada usuario, el identificador de la deuda padre que se obtiene como clave foránea de la tabla deudas y el identificador del usuario deudor que se obtiene como clave foránea de la tabla usuarios.

Tareas

Cada tarea tiene 4 propiedades: un identificador, un nombre, el identificador del componente al que pertenecen que se obtiene como clave foránea de la tabla componentes y un estado que es un enumerado que puede tomar los valores 'Pendiente', 'Asignada' y 'Terminada'.

Usuarios_encargados_tareas

Además al crear una tarea se seleccionan de la lista de invitados a los usuarios encargados de esta. Estos usuarios se añaden en la tabla usuarios_encargados_tareas que tiene dos propiedades: el identificador de la tarea que se obtiene como clave foránea de la tabla tareas y el identificador del usuario que se obtiene como clave foránea de la tabla usuarios.

Habitaciones

La tabla habitaciones tiene 4 propiedades: un identificador único, el nombre de la habitación, la capacidad máxima de la habitación y el identificador del componente al que pertenece, que se obtiene como clave foránea de la tabla componentes.

Camas

Cada cama en una habitación tiene un identificador único y pertenece a una habitación específica. La tabla camas tiene 3 propiedades: un identificador único, el nombre de la cama y el identificador de la habitación a la que pertenece, que se obtiene como clave foránea de la tabla habitaciones.

Cama_ocupada_por_usuario

La tabla cama_ocupada_por_usuario cuenta con dos propiedades, el identificador del usuario que ocupa la cama y el identificador de la cama. Ambas se obtienen como claves foráneas de las tablas usuario y camas respectivamente.

5.2 Servidor

El servidor es el núcleo de la aplicación, actúa como el intermediario entre los usuarios y la base de datos. Utiliza la biblioteca Socket.IO para manejar la comunicación en tiempo real entre los clientes y el servidor, permitiendo una interacción dinámica y fluida.

El servidor es responsable de procesar las solicitudes de los clientes, realizar las operaciones necesarias en la base de datos y enviar la información actualizada de vuelta a los clientes.

5.2.1 Inicio de la conexión con la aplicación

La comunicación entre la aplicación y el servidor se inicia cuando un usuario abre la aplicación. Al iniciar, se crea en esta una instancia de la clase `SocketService` que intenta establecer una conexión con el servidor con la URL 'http://viajesapp.ddns.net:3000/'. Esta dirección se mantiene constante gracias a NO IP, que permite mantener un nombre de dominio fijo incluso si la dirección IP pública del servidor cambia.

Cuando la conexión se establece correctamente, se dispara en la aplicación el evento 'onConnect'. En respuesta, la aplicación cambia el estado de la conexión a 'Online' y notifica a todos los listeners de este cambio. En caso de que la conexión se pierda por cualquier motivo, se dispara el evento 'onDisconnect' lo que provoca que el estado de la conexión cambie a 'Offline' y se notifique a todos los listeners. Más adelante se explicarán las repercusiones de este cambio de estado.

Este proceso establece una comunicación bidireccional entre el cliente y el servidor. El servidor almacena la información de la conexión con el cliente en la variable 'client', lo que permite identificar la conexión particular. A partir de este punto, el servidor escucha activamente el resto de los eventos que el cliente pueda disparar.

5.2.2 Manejo de eventos

La clase `socket.js` es la responsable de manejar la comunicación en tiempo real entre el servidor y los clientes utilizando la biblioteca Socket.IO. Esta clase define los manejadores de cada uno de los 39 eventos que puede disparar el cliente. Por lo general el disparado de un evento conlleva 3 acciones:

5.2.2.1 Creación, borrado y actualización en la base de datos:

Para cada evento disparado cuando un usuario intenta crear, borrar o actualizar la información de un usuario, viaje, componente o contenido de un componente, se llama desde el manejador del evento a las funciones definidas en la carpeta `Services` necesarias para realizar dichos cambios en la base de datos utilizando como parámetros de entrada la información recibida en el payload asociado al evento disparado. Los eventos que solo leen de la base de datos sin modificarla no tienen este paso.

Para cada objeto definido en el servidor, existe un conjunto de funciones que realizan la creación, borrado y actualización de la información correspondiente en la base de datos. Estas funciones, que ejecutan consultas SQL, se llevan a cabo de manera asíncrona para asegurarse de que la base de datos ha terminado todas las modificaciones antes de realizar cualquier operación de lectura. Una vez ejecutadas estas funciones pasamos al siguiente paso.

5.2.2.2 Operaciones de lectura en la base de datos y mapeo a objetos JS

Tras la ejecución de las operaciones de creación, borrado o actualización, o directamente si el evento es una solicitud de información, se ejecutan las funciones de lectura también definidas en la carpeta Services. Estas funciones leen los registros relevantes de la base de datos para generar los objetos que se definen en la carpeta 'Models' y se envían al cliente.

La carpeta "models" contiene los constructores de los objetos que se crean a partir de la información obtenida de la base de datos. Convertir esta información a objetos proporciona una estructura ordenada que facilita su posterior mapeo en objetos Dart dentro de la aplicación. En la carpeta "Models" se definen los constructores de las clases Viaje, Usuario, Componente y los distintos objetos que puede contener un componente.

Los objetos "Viaje" tienen las mismas propiedades que en la base de datos, aunque la fecha se transforma a una cadena de texto antes de enviarse al cliente.

```
Viaje {  
  id: '1', nombre: 'Viaje a Viena', fechaInicio: '2023-07-31',  
  fechaFin: '2023-08-05', estado: 'Planificando',  
  urlImagen: 'https://(...).JPG'  
}
```

Los objetos "Usuario" se emplean de dos maneras: para la creación de un nuevo usuario y su envío, y como componentes de mayor profundidad en dentro de otros objetos del sistema, como veremos en la descripción de la clase componente. En esos casos la contraseña se envía como vacía para no enviar información confidencial que no va a ser usada.

```
Usuario {  
  correo: 'bea@gmail.com',  
  contraseña: 'contraseña hasheada del usuario',  
  nombre: 'Beatriz', apellido_1: 'Sampedro',  
  apellido_2: 'Loro', color: 2  
}
```

Los objetos "Componente" son los más complejos. Aparte de las propiedades existentes en la base de datos, incluyen tres propiedades adicionales.

Dos de estas propiedades, llamadas propiedad_1 y propiedad_2 son de tipo dinámico y se utilizan para enviar información necesaria para la vista previa del componente en la pantalla del viaje en los componentes que lo necesitan. Por ejemplo, la primera propiedad podría ser el número de tareas del componente y la segunda el número de tareas completadas para que la tarjeta que representa el componente de tareas pueda mostrar una barra con el progreso. En caso de no recibir ningún valor se les asigna el valor int 0.

La tercera propiedad adicional es "subcomponente", que contiene una lista de objetos cuyos constructores están definidos en el resto de las carpetas de la carpeta "componentes" dentro de la carpeta "Models". La clase de los objetos que contenga esta lista depende de la propiedad tipo del componente, por ejemplo, un componente de confirmación de asistencia no puede contener objetos de la clase Tarea.

Estos objetos suelen tener además de sus las propiedades que tienen en la base de datos, una lista de otros objetos. Un ejemplo claro sería un componente de distribución de habitaciones que guarda en su propiedad "subcomponente" una lista de objetos "Habitación". Cada "Habitación" de la lista contiene, entre otras propiedades, una lista de objetos "Cama", que a su vez contienen entre otras propiedades, una lista de objetos "Usuario", representando a los usuarios que ocupan esa cama.

```
Componente {
  id: '559222c5-8842-4386-9d20-39f688fcdbcb',
  tipo: 'habitaciones', color: 2,
  subcomponente: [{Habitacion},{Habitacion},{Habitacion}],
  índice: 2, nombre: 'Cuartos', propiedad_1: 0,
  propiedad_2: 0
}
```

```
Habitacion {
  id: 'fdb554b4-65d8-4c8a-9717-213efdf500f4',
  nombre: 'Cuarto arriba derecha',
  camas: [{Cama},{Cama}]
}
```

```
Cama {
  id: 'fdb554b4-65d8-4c8a-9717-213efdf500f4',
  nombre: 'Cama doble 1',
  usuarios: [{usuario sin contraseña},{usuario sin contraseña}]
}
```


5.2.2.3 Envío de información actualizada a los clientes

Dependiendo de las necesidades de la aplicación, el disparo de un evento puede provocar que distintas cantidades de personas deban actualizar su información. Por ejemplo, si un usuario inicia sesión no necesitamos informar al resto, solo al cliente que ha enviado la solicitud. Para un cambio en la información de un viaje solo necesitamos informar a todos los miembros de dicho viaje que tengan la aplicación abierta y para invitar a un nuevo usuario a un viaje debemos actualizar la pantalla de un usuario distinto al que envía la petición. Para lograr esto he desarrollado 3 tipos de envío de información.

- **Envío al usuario que disparó el evento**

Cuando un cliente realiza una acción en la aplicación, como solicitar información de un viaje, dispara un evento específico en el servidor. El servidor, al recibir este evento, realiza las operaciones necesarias y devuelve la respuesta directamente al cliente que disparó el evento. Para ilustrar, en el caso del evento 'solicitud-viajes', el servidor obtiene la información del viaje y la devuelve al cliente que realizó la solicitud.

- **Envío a Rooms**

La biblioteca Socket.IO facilita la implementación de rooms o salas. Estos son canales a los que un cliente puede suscribirse y recibir mensajes enviados a ese canal específico.

Cuando un usuario inicia sesión, y pasa a la pantalla de inicio, dispara el evento "solicitud-viajes" para obtener una lista con todos los viajes a los que está invitado. Además de enviar al usuario la información solicitada, el servidor añade al cliente a los rooms asociados a todos sus viajes. Así, el cliente recibe en tiempo real todas las actualizaciones que otros usuarios realizan sobre el viaje mientras está utilizando la aplicación.

Cuando un usuario cierra la aplicación, se le elimina de todos los rooms a los que esté suscrito. Si algún room queda vacío tras esto, se elimina. Además, si se intenta añadir a un usuario a un room que no existe, el servidor crea automáticamente un nuevo room.

- **Envío a un cliente distinto al que disparó el evento**

Para poder invitar usuarios a un viaje y que estos vean el nuevo viaje al instante es necesario poder encontrar el canal de conexión entre dicho usuario y el servidor. Para desarrollar esta funcionalidad he implementado un mapa 'userToSocket'. Cuando el cliente se registra con un correo electrónico, la aplicación del usuario emite el evento 'register-user', pasando el correo electrónico como dato. El servidor que conoce el identificador de la conexión por la que ha llegado este evento, agrega una nueva entrada al mapa 'userToSocket', con el correo electrónico del usuario y la identificación del cliente.

Cuando un usuario invita a otro a un viaje introduciendo el correo del segundo en la aplicación, este correo se envía al servidor junto con el resto de información necesaria para añadir la invitación del usuario en la base de datos y para enviar al usuario invitado un evento que provoque que la información de su pantalla se actualice.

5.3 Aplicación

5.3.1 Información que se conserva en local

El diseño de la aplicación se basa en un enfoque de "conservación mínima de datos", manteniendo localmente solo la información estrictamente necesaria para las interacciones con el servidor. Este enfoque ayuda a garantizar que la información que se muestra a los usuarios esta siempre actualizada.

La información que se conserva localmente son los parámetros necesarios para las peticiones. Por ejemplo, cuando un usuario abre la pantalla de un viaje lo primero que hará la aplicación antes de mostrar nada es enviar una petición al servidor con el Id del viaje que se habrá guardado en local en el momento en el que el usuario pulsó el botón que lo llevó a esta pantalla, y no será sobrescrito hasta que el usuario entre en la pantalla asociada a otro viaje.

Esta información esencial se mantiene en cuatro campos dentro de la clase `SocketService`, permitiendo que cualquier pantalla pueda acceder y modificarlos según sea necesario, estos campos son:

- **Usuario:** cuando el usuario inicia sesión en la aplicación, el servidor envía junto a la confirmación todas las propiedades del usuario excepto su contraseña. Antes de llevar al usuario a la pantalla de inicio se guardan estas propiedades del usuario en el `SocketService`. Esta información es útil para saber que viajes mostrar en la aplicación y para los componentes que muestran información personalizada a cada usuario (como el componente de equipaje grupal que además de la lista común muestra una lista de los ítems que el usuario tiene asignados) entre otros casos.
- **Viaje:** cuando el usuario toca cualquier elemento de la aplicación que le vaya a llevar a la pantalla de un viaje, como la lista de viajes o el calendario, se guarda el id del viaje en el `SocketService` y después se mueve al usuario a la nueva pantalla.
- **Componente:** cuando el usuario toca cualquier elemento de la aplicación que lo lleve a una pantalla de componente, se guarda en el `SocketService` el id y el tipo de dicho componente y después se mueve al usuario a la nueva pantalla. Esto permite al servidor saber cuál es el componente que está siendo modificado ya que el id se adjunta a todos los payload relacionados con las modificaciones de componentes.
- **Estado de la conexión:** en el momento en el que la aplicación establece conexión con el servidor este campo tendrá valor `true`, si se pierde la conexión el valor vuelve a `false` e inutiliza todos los botones que envíen peticiones al servidor, además los vuelve grises y no interactivos (Figura 5.2) para que el usuario lo entienda fácilmente.



Figura 5.2 Botón de inicio antes y después de conectar con el servidor

5.3.2 Definición de objetos

La aplicación Flutter, similar al servidor, utiliza una carpeta de modelos para definir los objetos que se utilizan en la aplicación. Estos objetos siguen una estructura casi idéntica a la de los objetos del servidor, ya que están diseñados para ser creados a partir de los payloads que el servidor envía. Cada clase en esta carpeta tiene una función `from.Map` diseñada para recibir como parámetro de entrada el mapa contenido en el payload que acompaña a los eventos de actualización de pantalla.

5.3.2.1 Viajes y usuarios

Los objetos de las clases viaje y usuario se construyen a partir de la información recibida en el payload sin aplicar ningún cambio en su estructura a excepción de las fechas del viaje convierten de string al formato de fecha de Flutter y del color del usuario que llega como un índice representando una posición en una lista de colores definida en la clase usuario de la aplicación, por lo que en la propiedad color del usuario guardamos el valor del color en lugar de la posición en la lista.

5.3.2.2 Componentes

Los componentes se construyen de manera diferente, ya que implementan herencia de clases. Cada tipo de componente (como `ComponenteHabitacion`, `ComponenteAsistencia`, `ComponenteDeudas`, etc.) hereda de una clase base `Componente`. La clase base `Componente` tiene una función de fábrica `Componente.fromMap` que toma la información del payload, lee el tipo del componente y según su valor elige a cuál de los constructores de las subclases de componente enviar la información para crear el nuevo objeto. La principal diferencia entre componentes es la clase de objetos que contiene su propiedad subcomponente, en el constructor del componente se llama a la función `.from.Map` de la clase que queremos construir. Por ejemplo, para un componente de tareas cuyo subcomponente es una lista de objetos de clase `Tarea` se utiliza la función “`Tarea.fromMap`” que recibe cada uno de los objetos JavaScript tarea enviados en la propiedad subcomponente del payload.

5.3.2.3 Objetos dentro de los componentes

Los objetos dentro de los componentes se construyen de forma similar a viajes y usuarios con la diferencia de que la mayoría de estos contienen otros objetos, por lo que en su propia función `from.Map` llaman a la función `from.Map` de otras clases. Por ejemplo, un objeto de la clase `tarea` que contiene varios objetos de la clase `usuario` a modo de lista de usuarios asignados.

5.3.3 Implementación de las pantallas

En Flutter todos los elementos que se muestran en pantalla son Widgets. Los widgets son los bloques de construcción básicos de la interfaz de usuario. Cada pantalla de la aplicación es un widget que, al igual que el resto, puede contener otros widgets de texto, botones, o columnas entre otros muchos.

Las pantallas de esta aplicación son statefull widgets, esto significa que pueden cambiar, ya sea debido a interacciones del usuario (como tocar un botón) o cambios en los datos (como recibir una actualización del servidor).

Para dibujar un statefull widget es necesario tener definido un método llamado `initState` que se ejecuta automáticamente cada vez que el usuario entra en la pantalla. Este método inicializa las variables de estado del widget y ejecuta el método `build` que es el encargado de dibujar en pantalla todos los componentes programados leyendo cuando se considera necesario las variables de estado. El "estado" en Flutter se refiere a la información que puede cambiar, por ejemplo, un widget texto que muestre el valor de un contador, el valor del contador sería considerado el estado de ese widget, al ejecutar la función `initState` se inicializa esa variable con el valor definido en el código y después se dibuja el widget.

Las variables de estado no se actualizan en tiempo real en la pantalla, por ejemplo, en el caso del contador si hubiese un botón bajo el número que al pulsarse sumase uno al valor del contador y lo imprimiese por la consola, veríamos que, al pulsarlo, el número va incrementando en la consola pero no en la pantalla de la aplicación.

Esto se debe a que el método `build` ya ha leído las variables de estado y ha terminado de dibujar los widgets. Una vez termina de dibujar acaba su ejecución. Para actualizar la información en pantalla es necesario llamar a la función `setState` que llama de nuevo al método `build` para que redibuje la pantalla con las variables de estado actualizadas.

```
setState(() {contador= contador+1});
```

En esta aplicación la información que nos interesa guardar en las variables de estado son los viajes a los que está invitado el usuario, los componentes que pertenecen a esos viajes y el contenido de dichos componentes.

Esta información no se almacena en local, es decir, si un usuario que ha recibido la información actualizada de todos sus viajes pierde la conexión al servidor, en el momento en que salga de la pantalla de viajes e intente volver no verá nada ya que esta información solo se guarda en el estado de la pantalla. Además las modificaciones a las variables de estado tampoco se ejecutan en local, cuando un usuario añade un nuevo componente a un viaje no está añadiendo un nuevo objeto de la clase componente a la lista de componentes almacenada en el estado de su pantalla, en realidad está disparando un evento "creación-componente" que se envía al servidor junto a un payload que contiene la información que el usuario haya dado para crear el componente, y la información necesaria almacenada en el `SocketService` que como ya explique, es la única que se conserva entre pantallas. Por ejemplo, para crear un componente se envía en el payload un mapa con las propiedades tipo, color y nombre que el usuario ha proporcionado para crear el componente, y el id del viaje que se obtiene del `SocketService`, para saber a qué viaje añadir el nuevo componente y a que usuarios enviar la actualización de la información.

Este sistema de actualización implica que el `initState` además de inicializar las variables de estado y llamar al método `build`, también tiene que lanzar un evento al servidor para solicitar la información que guardar en las variables de estado, también debe definir los manejadores de los eventos asociados a la respuesta del servidor para poder recibir dicha información actualizada, mapearla a objetos Dart, asignar los valores de dichos objetos a las variables de estado y llamar al método `setState` para redibujar la pantalla con la información actualizada. Además, el método `initState` mantiene la suscripción a los eventos de recibo de información ya que la pantalla no solo se actualiza al abrirla, se actualiza cada vez que cualquier usuario con acceso a esa pantalla modifica la en la base de datos la información asociada a esta como en el ejemplo de crear un componente que he mencionado antes.

5.3.4 Planteamiento de las pantallas

5.3.4.1 Pantalla de Login

Al abrir la aplicación el usuario se encuentra con la pantalla de Login que puede verse en la Figura 5.4, en la que debe rellenar los campos de correo y contraseña para poder iniciar sesión.

Hasta que la aplicación pueda conectarse con el servidor el botón de Ingresar se muestra de color gris y no reacciona al ser pulsado (Figura 5.3), cuando la aplicación logra conectarse se vuelve del color naranja como vimos en la Figura 5.4, al ser pulsado envía al servidor el usuario y la contraseña hasheada para comprobar en la base de datos si la información es correcta. Si la información enviada es correcta se notifica a la aplicación, que llevará al usuario a la pantalla de inicio. En caso de error en la contraseña/correo se muestra al usuario un mensaje en rojo informándole de que ha introducido mal sus datos.

Además, el campo de correo marcará un mensaje en rojo si el correo introducido no tiene un formato válido, es decir, que su estructura no es texto + arroba + texto + . + al menos dos letras como podemos ver en la Figura 5.5. De igual forma la contraseña mostrara un mensaje de error si no se han introducido al menos 6 caracteres que es la longitud mínima de una contraseña en la aplicación.

Debajo del botón de Ingresar hay un segundo botón que lleva al usuario a la pantalla de creación de cuenta que se explicará en detalle a continuación.

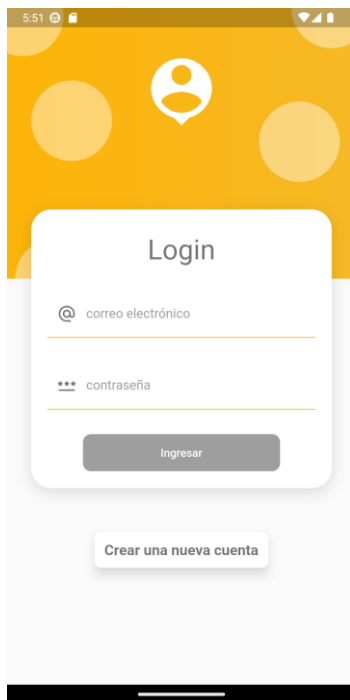


Figura 5.3 Pantalla de Login sin conexión al servidor

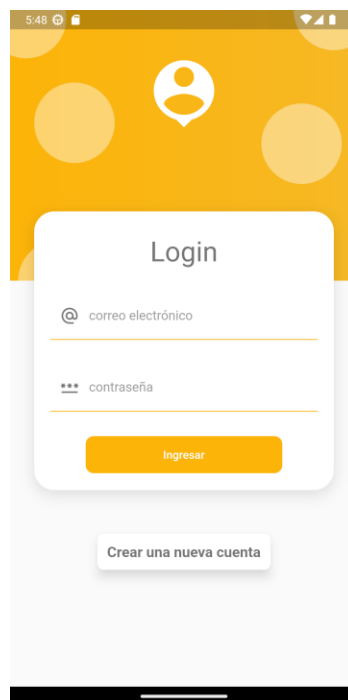


Figura 5.4 Pantalla de Login con conexión al servidor

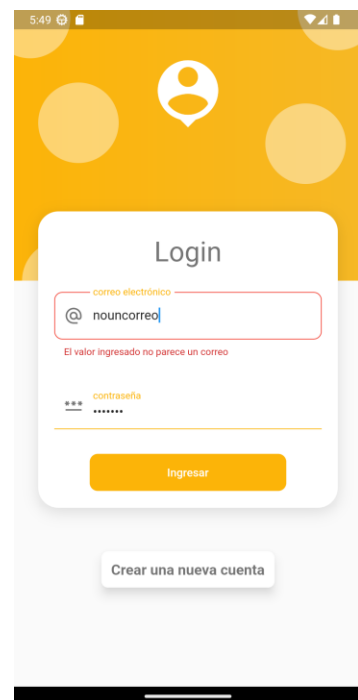


Figura 5.5 Pantalla de Login mostrando error de formato en el correo

5.3.4.2 Pantalla de creación de cuenta

La pantalla de creación de cuentas que puede verse en la Figura 5.6, muestra una serie de campos de texto a rellenar por el usuario seguidos de una lista de colores entre los cuales el usuario debe elegir uno que será el color de su perfil.

Los campos de nombre, primer apellido y segundo apellido muestran el teclado con la primera letra en mayúsculas para facilitar la entrada de datos, el campo de email abre el teclado en minúsculas y muestra también la fila de números en la parte superior de este y el arroba en la parte inferior junto a la tecla de espacio. Al pulsar el botón “Crear” el usuario es llevado de nuevo a la pantalla de Login para que pueda introducir sus datos e iniciar sesión.

Al igual que la pantalla anterior, la pantalla de creación de cuentas cuenta con un sistema de detección de errores que marca en rojo los campos con algún error como puede verse en la Figura 5.7.

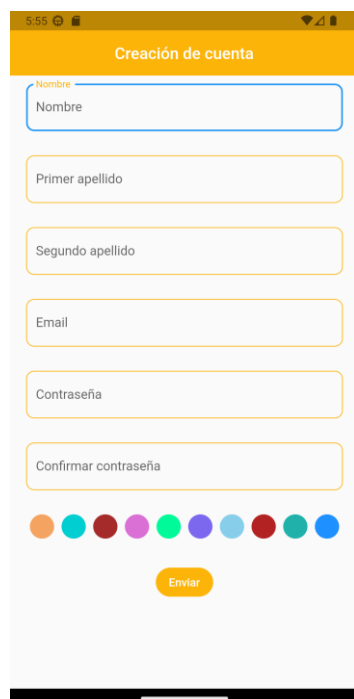


Figura 5.6 Pantalla de creación de cuenta vacía

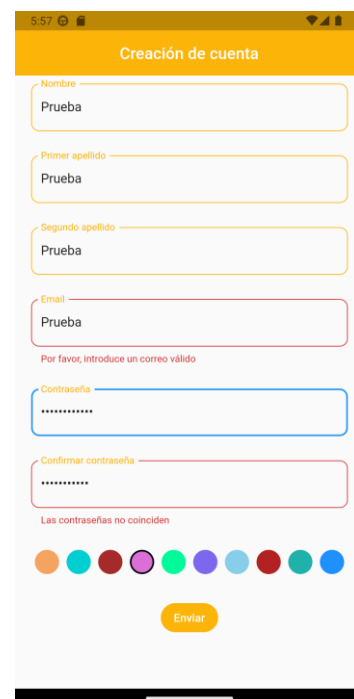


Figura 5.7 Pantalla de creación de cuenta después de rellenar los campos, muestra errores en los datos

5.3.4.3 Pantalla de inicio

Después de iniciar sesión los usuarios son enviados a la pantalla de inicio. Esta pantalla muestra al usuario su próximo viaje más cercano, sus viajes próximos y pasados y un calendario que muestra los rangos de fechas de todos los viajes en los que participa el usuario (Figura 5.8 y Figura 5.9). Además si se da el caso de que el usuario participe en varios viajes que se solapen, se indica marcando los días solapados en rojo para que el usuario sepa que algo va mal (Figura 5.12). Tocar un día marcado en naranja lleva al usuario a la pantalla del viaje asociado a esa fecha, tocar un día marcado en rojo abrirá un pop-up con una lista de nombres los viajes que ocupan ese día (Figura 5.13). Tocar cualquiera de esos nombres lleva a usuario al viaje correspondiente para que pueda gestionar el problema.

Además los usuarios pueden crear viajes nuevos pulsando el botón flotante de la esquina inferior derecha y dando un nombre para el viaje, una URL a una imagen y un rango de fechas (Figura 5.10 y Figura 5.11). Al crear el viaje este se añadirá al momento a la pantalla de inicio.

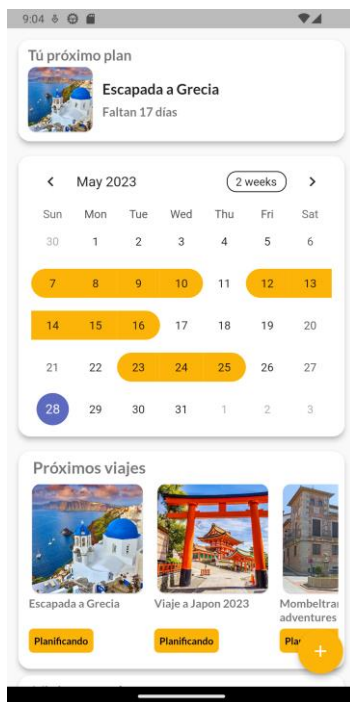


Figura 5.8 Pantalla de inicio parte superior

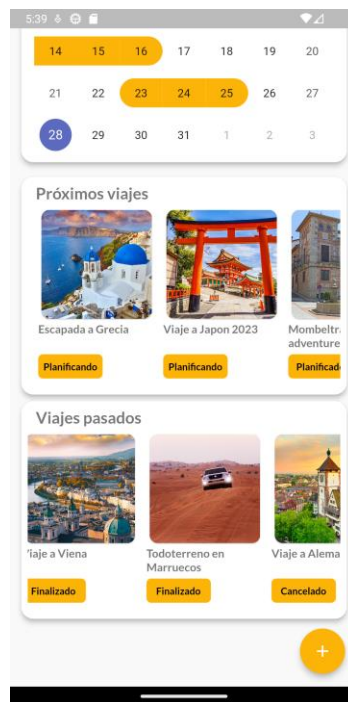


Figura 5.9 Pantalla de inicio parte inferior

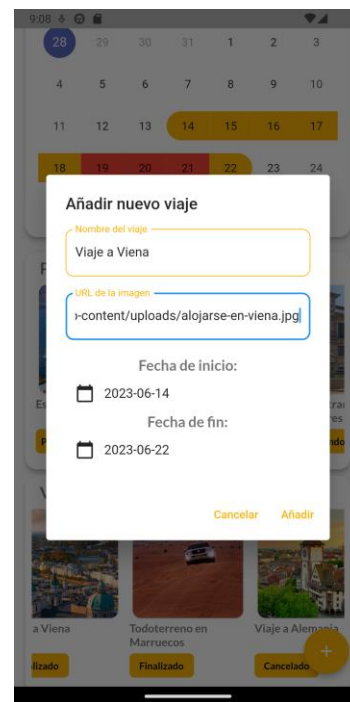


Figura 5.10 Pop up para añadir un nuevo viaje

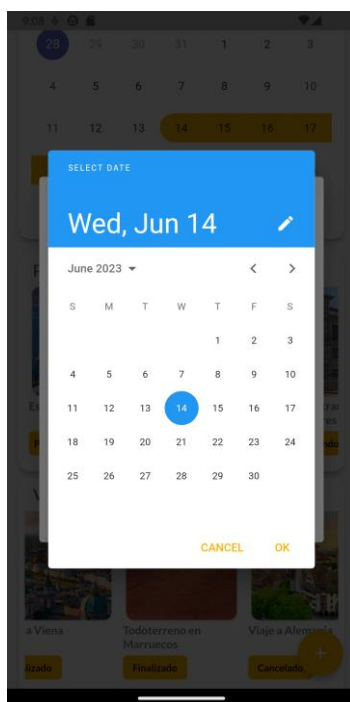


Figura 5.11 Pop up para elegir fechas para el viaje

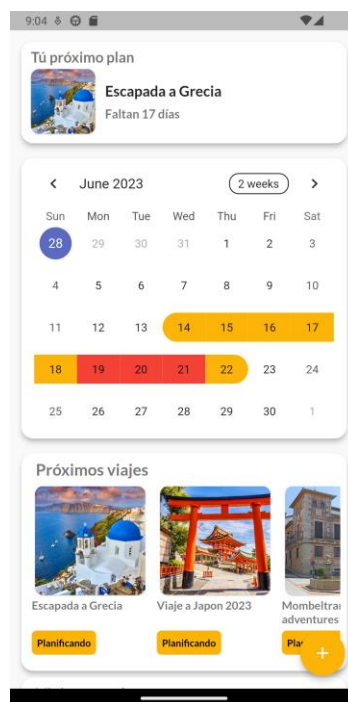


Figura 5.12 Calendario con viajes solapados

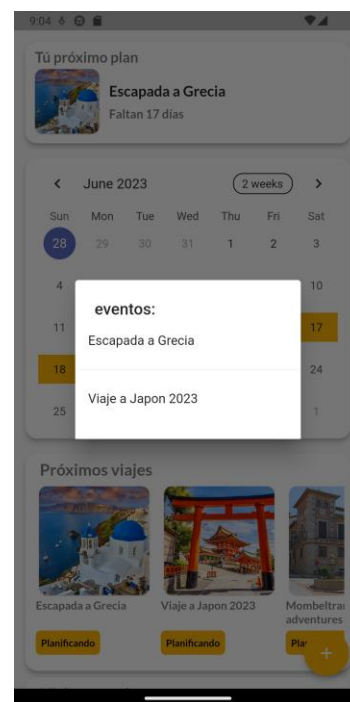


Figura 5.13 Pop up para seleccionar uno de los viajes solapados

5.3.4.4 Pantalla de viaje

A un viaje podemos añadir 7 tipos de componente, más adelante explicaremos cada uno de ellos en profundidad. Para crear un nuevo componente el usuario pulsar el botón con el símbolo “+” de la esquina inferior derecha, lo que abrirá un pop up en el que debe seleccionar de la lista desplegable el tipo de componente que quiere crear, dar un nombre para el componente y elegir de la lista de colores el que mejor le parezca. Seleccionar el tipo de componente y el color cambian en tiempo real la vista previa del icono del componente que se encuentra a la derecha del desplegable de tipos. Este Pop up puede verse en la Figura 5.16

Las tarjetas de componentes no son solo un vínculo entre la pantalla de viaje y la pantalla de cada componente, están diseñadas para mostrar de forma resumida y clara la información más relevante del componente que representan, de modo que la lista de tarjetas actúa como panel de control permitiendo a los usuarios ver fácilmente el estado de la planificación del viaje sin tener que abrir cada uno de los componentes que contiene. En la Figura 5.14 y la Figura 5.15 puede verse como cambian las tarjetas a medida que avanza la planificación. Cada tipo de componente tiene asociado un tipo de tarjeta diseñado para mostrar de la mejor forma posible la información más relevante.

Sobre el botón de añadir componente hay un segundo botón que abre un Pop up que permite a los invitados de un viaje añadir más invitados, eliminarlos, modificar las fechas de inicio y fin del viaje, cambiar el estado del viaje, su nombre y borrar el viaje entero. Este Pop up puede verse en las figuras : Figura 5.17, Figura 5.18 y Figura 5.19.



Figura 5.14 Pantalla de viaje con componentes vacíos



Figura 5.15 Pantalla de viaje después de modificar los componentes

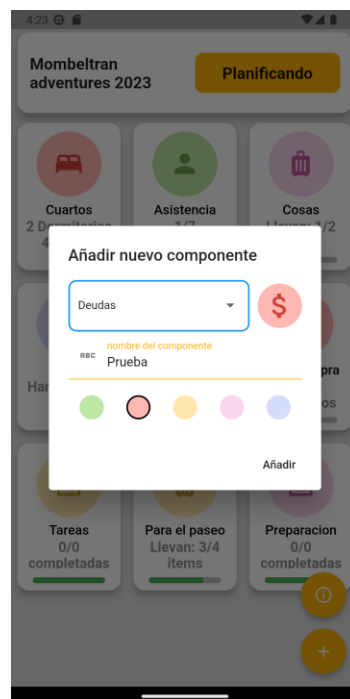


Figura 5.16 Pop up para añadir nuevos componentes

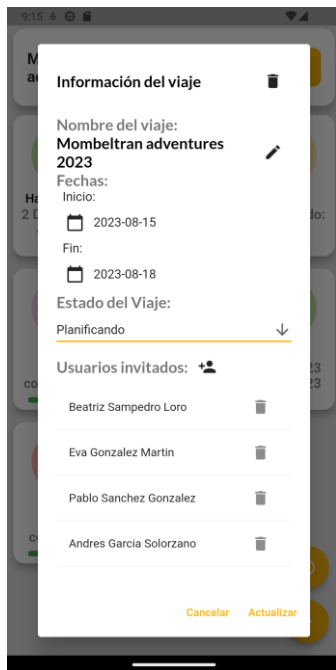


Figura 5.17 Pop up para actualizar informacion de un viaje e invitar usuarios

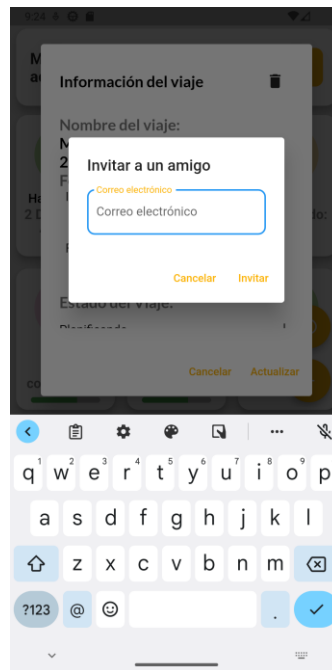


Figura 5.18 Pop up para invitar usuarios al viaje

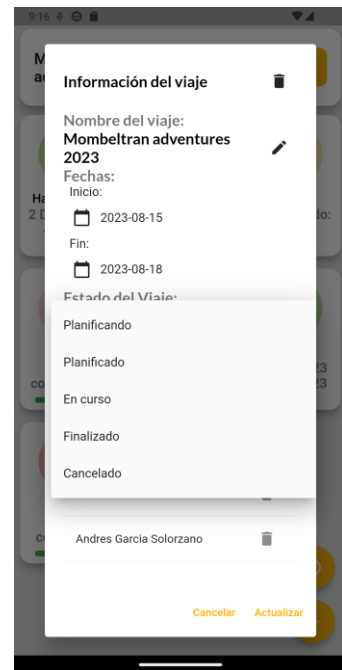


Figura 5.19 Desplegable para actualizar el estado del viaje

5.3.4.5 Componente de distribución de habitaciones

Este componente proporciona una representación gráfica y detallada de la distribución de habitaciones y camas para facilitar su asignación a los usuarios (Figura 5.21). Con frecuencia, en páginas de alquiler como Airbnb, se anuncia únicamente el número total de camas, obligando a los usuarios a interpretar la distribución a través de fotos, lo cual puede resultar confuso y generar malentendidos. Este componente soluciona estos inconvenientes, presentando forma visual la distribución y asignación de habitaciones y camas en la propiedad.

Los usuarios pueden añadir nuevas habitaciones (Figura 5.20) y dentro de estas nuevas camas, además pueden añadirse o eliminarse a sí mismos de estas (Figura 5.22).



Figura 5.20 Pop up para añadir una habitación

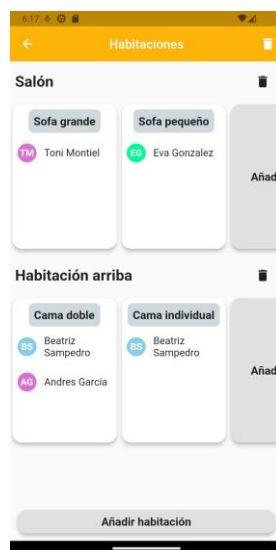


Figura 5.21 Interfaz del componente

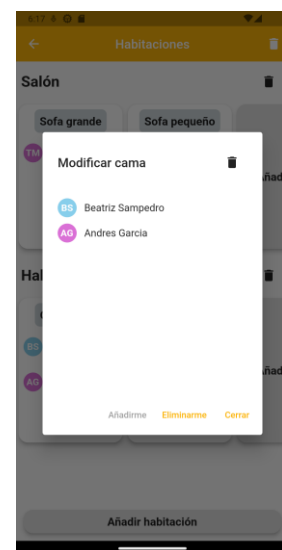


Figura 5.22 Pop up para añadirse o borrarse de la cama

5.3.4.6 Componente de confirmación de asistencia

El componente de asistencia permite a los usuarios indicar si asistirán o no a un viaje. La pantalla está formada por 3 listas que muestran a los usuarios repartidos según su estado de confirmación, estas listas pueden verse en la Figura 5.23 y en la Figura 5.25. Para actualizar el estado de su confirmación los usuarios pueden pulsar el botón de la esquina inferior derecha que despliega un pop up en el que elegir entre los estados “Pendiente”, “Confirmado” y “No asistirá” (Figura 5.24). Cuando se crea un componente de asistencia se crean automáticamente las confirmaciones de todos los invitados al viaje con estado “Pendiente” por defecto. Borrar o invitar nuevos usuarios al viaje elimina y añade confirmaciones a la lista.

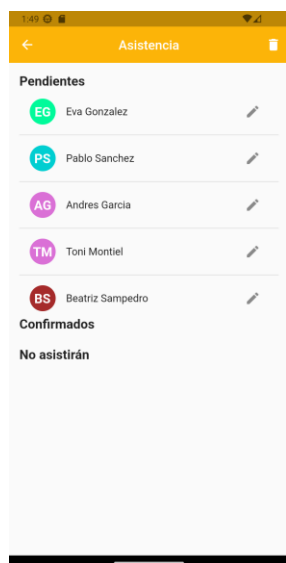


Figura 5.23 Pantalla de confirmación sin modificar

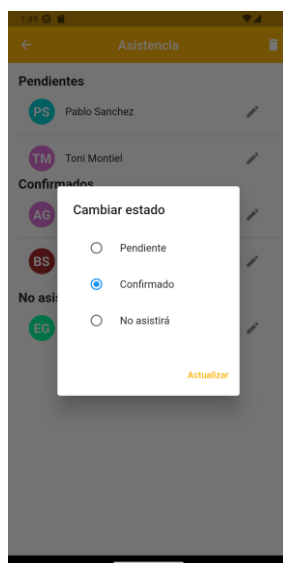


Figura 5.24 Pop up para actualizar estado de la confirmación

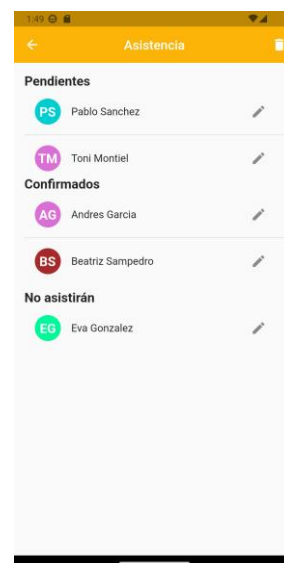


Figura 5.25 Pantalla de confirmación tras ser modificada

5.3.4.7 Componente de deudas

El componente de deudas permite a los usuarios mantener un registro de los pagos que realizan durante el viaje y saldar fácilmente las deudas asociadas a estos.

Para añadir un nuevo gasto el usuario debe pulsar el botón de la parte inferior derecha de la pantalla lo que abrirá un Pop up donde añadir la información necesaria sobre el gasto incluyendo el nombre de la deuda, la cantidad, el usuario acreedor y los deudores (Figura 5.26). Una vez añadido, el gasto será visible para todos en la lista de deudas indicando a la derecha de cada una el dinero que el usuario debe recibir o pagar por ese gasto (Figura 5.27). Si el usuario no ha participado en el gasto, la parte derecha de este queda vacía.

Los usuarios pueden consultar su balance total pulsando el botón “balance” que cambia el contenido de la pantalla para mostrar la cantidad de dinero que deben o que se les debe junto a una lista de deudas simplificadas de todos los usuarios (Figura 5.28). Las deudas simplificadas representan la forma más sencilla para los usuarios de pagar el dinero que deben con el menor número de transferencias posibles. Tras saldar una deuda los usuarios pueden pulsar el icono de tarjeta de la deuda simplificada, este icono abre el pop up “pagar deudas” (Figura 5.29) que permite al usuario indicar que ha pagado su deuda creando una deuda con la cantidad debida al acreedor con el nombre “Pago deuda” (Figura 5.31).

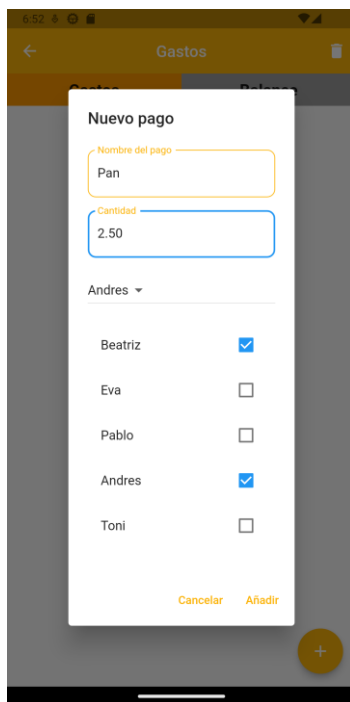


Figura 5.26 Pop up para añadir una nueva deuda

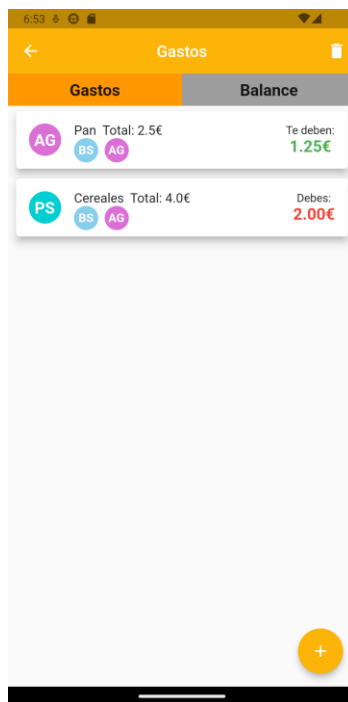


Figura 5.27 Lista de deudas

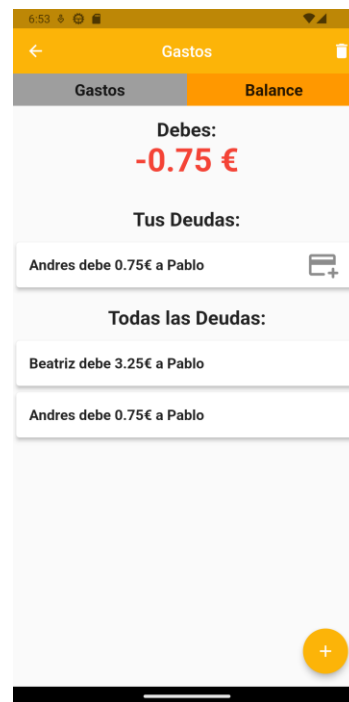


Figura 5.28 Balance del usuario y deudas simplificadas

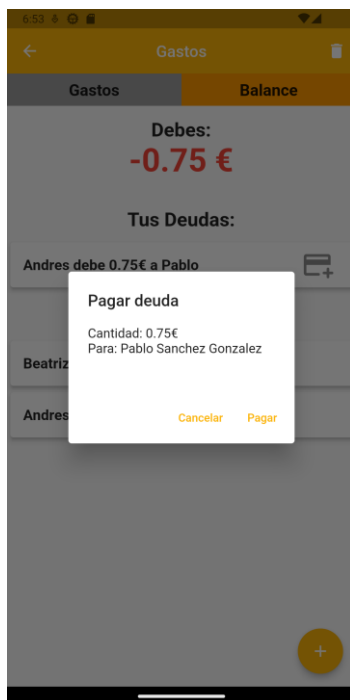


Figura 5.29 Pop up para saldar todas las deudas con una persona

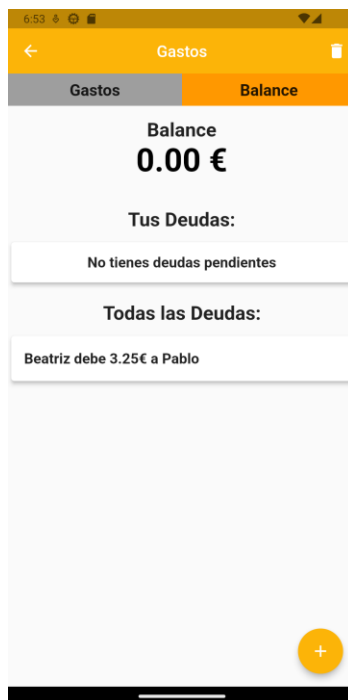


Figura 5.30 Balance en equilibrio

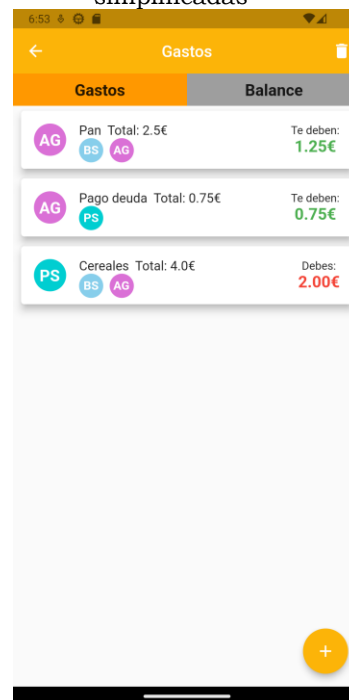


Figura 5.31 Lista de deudas después del pago

5.3.4.8 Componente de equipaje grupal

En el componente de equipaje grupal cualquier usuario invitado al viaje puede añadir con el botón de la esquina inferior derecha un nuevo ítem grupal. Pulsar este botón abrirá un pop up que pedirá al usuario un nombre y una cantidad para el nuevo ítem (Figura 5.32). Al abrir el pop up se despliega automáticamente el teclado con la primera tecla en mayúscula y apuntando al campo nombre, al tocar en el campo cantidad el teclado pasa a ser numérico.

Cuando el usuario pulsa en añadir el ítem se añade al instante a la pantalla en la lista de ítems comunes. Los usuarios pueden tocar cualquiera de estos ítems para desplegar un pop up en el que podrán modificar la cantidad requerida de dicho ítem y la cantidad del mismo encargado a cada usuario invitado al viaje pulsando los botones de + y - situados junto a los nombres (Figura 5.33).

Debajo de la lista de ítems comunes tenemos una lista llamada “Que llevas tú” que muestra los ítems para los que el usuario usando la aplicación tiene asignada una cantidad mayor a 0. Se muestra el nombre del ítem y la cantidad asignada como puede verse en la Figura 5.34. Esto permite al usuario ver fácilmente lo que debe llevar al viaje sin tener que revisar cada ítem de forma individual.

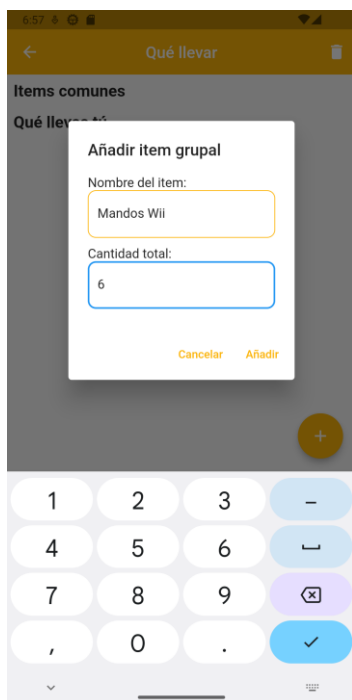


Figura 5.32 Pop up para añadir un ítem grupal

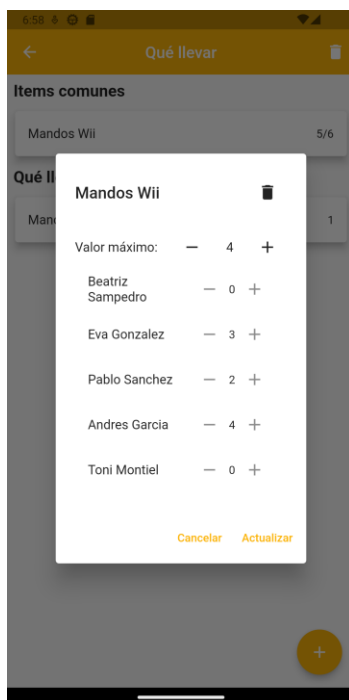


Figura 5.33 Pop up para asignar ítems y actualizar la cantidad necesaria

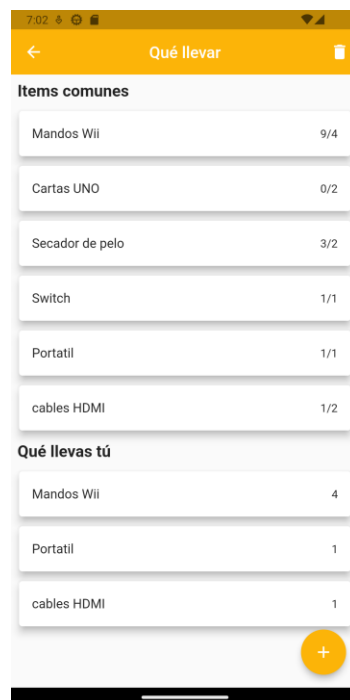


Figura 5.34 Lista de ítems grupales con resumen de lo encargado al usuario

5.3.4.9 Componente de tareas

En este componente podemos ver dos listas de tareas (Figura 5.37), una para las tareas pendientes y otra para las completadas. Cualquier usuario invitado al viaje puede añadir nuevas tareas pulsando el botón de la esquina inferior derecha. Pulsar este botón abrirá un pop up en el que añadir el nombre y encargados de la tarea para añadirla a la lista (Figura 5.35). Las tareas se añaden inicialmente a la lista de pendientes, aquellas tareas que se terminen pueden cambiarse su estado pulsando el icono situado en la parte derecha de la tarea para abrir un pop up en el que marcar la tarea como terminada (Figura 5.36) al actualizarse se moverán a la lista de tareas completadas.

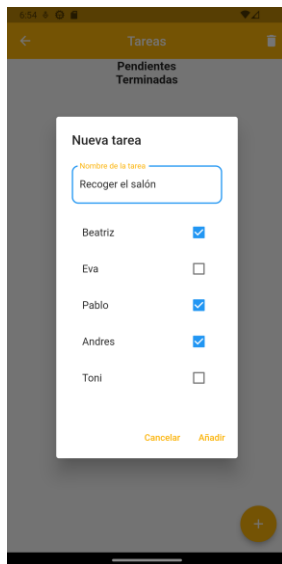


Figura 5.35 Pop up para añadir tareas



Figura 5.36 Pop up para actualizar el estado de una tarea

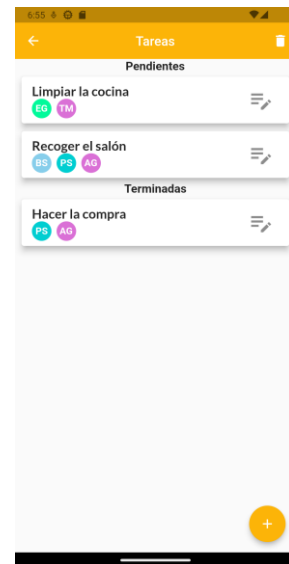


Figura 5.37 Listas de tareas

5.3.4.10 Componente de compra

El componente de compra permite a los usuarios tener una lista de productos actualizada en tiempo real (Figura 5.41). Cualquier invitado al viaje puede añadir productos a la lista usando un pop up en el que escribir el nombre del producto (Figura 5.38). Cuando los encargados de hacer la compra vayan a la tienda pueden ir marcando los productos de la lista como comprados deslizándolos a la derecha lo que los mueve a la lista de comprados (Figura 5.39). Además si por cualquier motivo un usuario desea eliminar el ítem de la lista antes de que sea comprado puede deslizarlo hacia la dirección contraria para eliminarlo (Figura 5.40). Antes de eliminarlo saltará un mensaje de confirmación. Gracias a la actualización en tiempo real de la información los encargados de comprar pueden sincronizarse y el resto de los usuarios pueden añadir en cualquier momento nuevo productos.

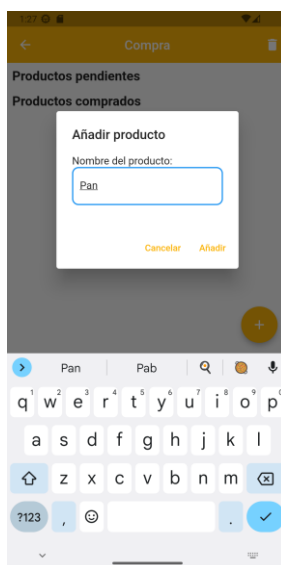


Figura 5.38 Pop up para añadir producto a la lista

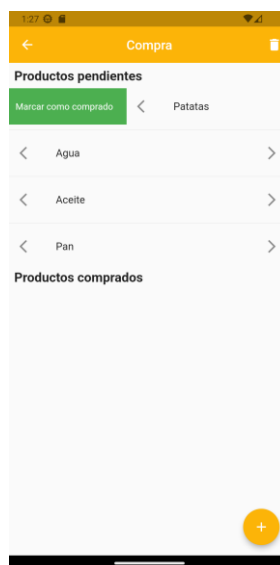


Figura 5.39 Ítem marcándose como comprado

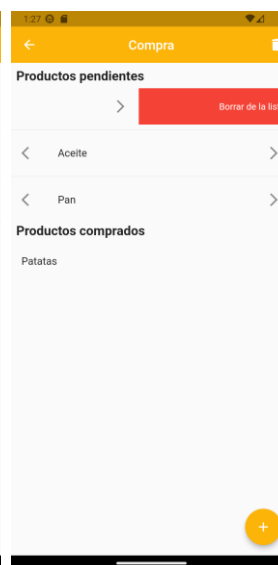


Figura 5.40 Ítem siendo borrado

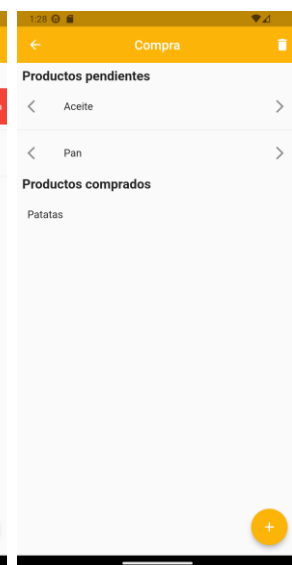


Figura 5.41 Lista de la compra

5.3.4.11 Componente de fechas

El componente de fechas está diseñado para ayudar a los usuarios a encontrar fechas en las que todos estén disponibles para el viaje. En el calendario de este componente se muestran dos cosas:

- **Fechas del viaje:** El calendario muestra en verde el rango de fechas seleccionado para el viaje, que puede ser creado o actualizado pulsando el botón azul en la parte inferior del calendario (Figura 5.42).
- **Eventos:** Además podremos ver marcadas en rojo las fechas en las que cualquiera de los invitados al viaje no podría asistir. Estas fechas pueden ser añadidas por cada usuario pulsando el botón de la parte inferior derecha de la pantalla que abre un pop up preguntando por la fecha inicial y final del rango en que no podrán acudir. En la parte inferior de la pantalla se encuentra una lista con los eventos del usuario con un icono que puede pulsarse para borrar el evento (Figura 5.43).

Debajo del calendario hay una lista con todos los invitados al viaje en la que podemos desmarcar a los usuarios cuyas fechas no disponibles queramos ocultar en caso de que no sea posible elegir una fecha disponible para todos (Figura 5.44).

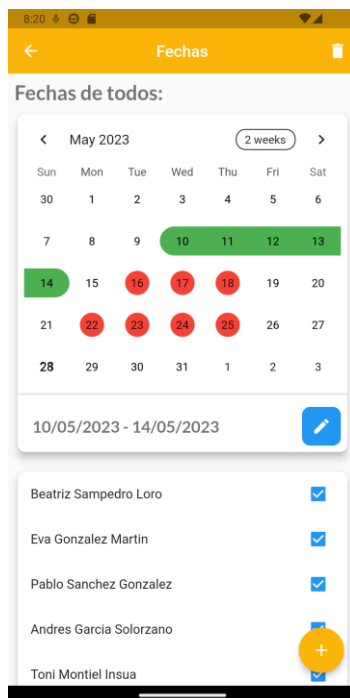


Figura 5.42 Pantalla de fechas con fechas finales y eventos

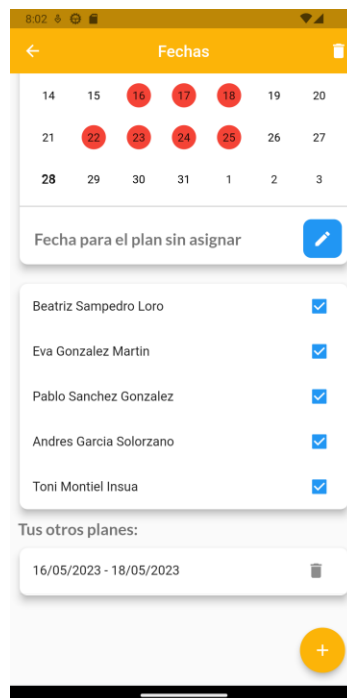


Figura 5.43 Parte inferior de la pantalla

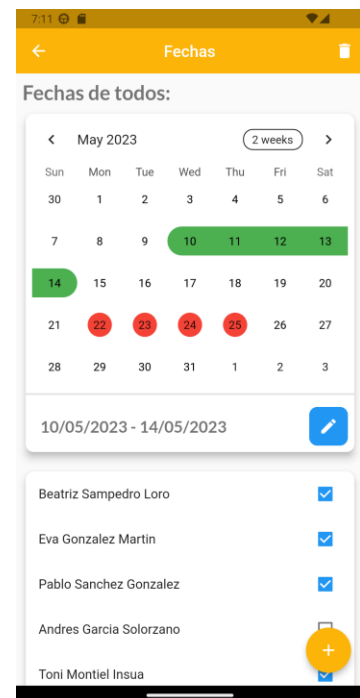


Figura 5.44 Pantalla de fechas con los eventos de un usuario ocultos

6 Evaluación y futuros cambios

En este capítulo se muestra una tabla con los resultados de las pruebas con los usuarios medidas en segundos. Después una lista con los problemas encontrados durante las pruebas y una segunda lista con otros cambios que podrían ser implementados en el futuro para mejorar la aplicación.

6.1 Pruebas con usuarios

Para evaluar la aplicación se realizaron una serie de pruebas con 5 usuarios. Estas pruebas se dividen en 8 categorías: la primera se refiere a las pruebas del sistema principal y las siete restantes corresponden a cada uno de los 7 tipos de componentes que pueden incorporarse a un viaje.

El rendimiento en las pruebas se midió en función del tiempo necesario para completarlas, los resultados están expresados en segundos sin decimales. Con el objetivo de evaluar cuán intuitivo es el sistema, antes de cada prueba se explicó al usuario el objetivo a cumplir, sin detallar en ningún momento los pasos que tendría que seguir para lograrlo. Por ejemplo, para la prueba "confirmar asistencia", las indicaciones para cada usuario fueron "para la siguiente prueba tienes que confirmar que asistirás al viaje", en lugar de decir "para la siguiente prueba tendrás que tocar el ícono del lápiz junto a tu nombre y seleccionar la opción confirmado".

Las pruebas con los usuarios se llevaron a cabo de manera presencial en sesiones individuales a lo largo de dos semanas. Una vez iniciadas las pruebas con cada usuario, el proceso no se detenía hasta finalizar.

El proceso de evaluación fue el mismo para todos los usuarios: después de descargar e instalar la APK en su dispositivo se explicaba el propósito y la utilidad de la aplicación. Una vez entendido el concepto de la aplicación, daban comienzo las pruebas. Antes de cada prueba, se indicaba al usuario el objetivo a lograr, y una vez listo, se iniciaba un cronómetro que se pausaba al alcanzarse dicho objetivo.

Las pruebas realizadas fueron las siguientes:

- Sistema
 - Crear cuenta
 - Iniciar sesión
 - Crear viaje
 - Invitar a otro usuario al viaje
 - Crear un componente
- Componente de distribución de habitaciones
 - Añadir una cama
 - Añadirse a la cama
 - Borrar el componente
- Componente de confirmación de asistencia
 - Confirmar asistencia
 - Cambiar estado
- Componente de gestión de gastos
 - Añadir un gasto
 - Comprobar el balance
 - Añadir un segundo gasto
 - Salvar la deuda
- Componente de lista de la compra

- Añadir dos productos
- Eliminar un producto
- Marcar producto como comprado
- Componente de tareas
 - Añadir una tarea
 - Marcar tarea como terminada
 - Borrar la tarea
- Componente de equipaje grupal
 - Crear un ítem
 - Modificar cantidad del ítem
 - Asignar cantidad de ítem
 - Eliminar el ítem
- Componente de fechas
 - Marcar días no disponibles
 - Ocultar días no disponibles
 - Mostrar días de nuevo
 - Seleccionar fecha para viaje

6.1.1 Resultados de las pruebas

La Tabla 1 muestra los resultados (en segundos) de las pruebas realizadas con cada uno de los cinco usuarios. Las celdas cuyo contenido está marcado en rojo indican un problema en la prueba que se explicarán más adelante.

Tabla 1 Pruebas con los usuarios medido en segundos

	Acción	Usuario 1	Usuario 2	Usuario 3	Usuario 4	Usuario 5
General	Crear cuenta	44	47	51	40	60
	Iniciar sesión	10	11	19	9	14
	Crear viaje	156	165	175	145	Error
	Invitar a otro usuario al viaje	20	24	26	18	28
	Crear un componente	14	19	21	18	25
Componente de distribución de habitaciones	Añadir una habitación	7	10	8	6	9
	Añadir una cama	4	5	6	3	8
	Añadirse a la cama	4	8	7	6	7
	Borrar el componente	4	19	5	5	25
Componente de confirmación de asistencia	Confirmar asistencia	9	11	13	8	12
	Cambiar estado	2	3	4	1	4
Componente de gestión de gastos	Añadir un gasto	15	19	20	14	23
	Comprobar el balance	7	9	11	6	13
	Añadir un segundo gasto	10	14	17	9	26
	Saldar la deuda	10	12	14	8	16
Componente de lista de la compra	Añadir dos productos	8	11	13	7	15
	Eliminar un producto	8	3	4	3	6
	Marcar producto como comprado	2	1	1	1	4
Componente de tareas	Añadir una tarea	11	14	15	10	20
	Marcar tarea como terminada	3	5	4	4	6

	Borrar la tarea	4	6	8	3	10
Componente de equipaje grupal	Crear un ítem	6	9	9	11	13
	Modificar cantidad del ítem	5	8	9	5	11
	Asignar cantidad de ítem	4	7	9	3	11
	Eliminar el ítem	4	6	6	3	11
Componente de fechas	Marcar días no disponibles	15	Error	21	Error	Error
	Ocultar días no disponibles	4	6	6	5	6
	Mostrar días de nuevo	1	2	1	1	2
	Seleccionar fecha para viaje	14	18	20	13	22

6.1.2 Problemas encontrados

- Todos los usuarios apuntaron que tener que introducir un enlace para la imagen de los viajes era un proceso incomodo y que preferirían poder seleccionar una imagen de la galería. Aproximadamente $\frac{3}{4}$ del tiempo dedicado a la creación del viaje se invertían en buscar un enlace con el formato correcto, el usuario 5 no lo logró pasados 3 minutos por lo que se pasó a la siguiente prueba.
- A la hora de borrar un componente los usuarios 2 y 5 intentaron mantenerlo pulsado para arrastrarlo imaginando que se podrían eliminar arrastrándolo a una papelera, este proceso de hecho era la idea original para el sistema de borrado pero por limitaciones de tiempo no pudo ser desarrollado. Los usuarios 1, 3 y 4 se fijaron en el icono de papelera de la pantalla del componente durante las pruebas anteriores y se dirigieron directamente a este. En el futuro debería implementarse el sistema de borrado que los usuarios 2 y 5 probaron ya que resulta más intuitivo.
- En el componente de habitaciones varios usuarios apuntaron que preferirían poder añadir y eliminar al resto de usuarios de las camas en lugar de solo poder moverse a sí mismos
- En el componente de la compra el primer usuario intento tocar los ítems de la lista a modo de botón antes de arrastrarlos a izquierda y derecha, al verlo añadir iconos de flecha a ambos lados de cada ítem para indicar que se interactúa con ellos deslizándolos, el resto de los usuarios no tuvieron problemas a excepción de uno que intento usar el icono de botón pero rectificó rápidamente.
- El componente de fechas resulta confuso para todos los usuarios, entienden su utilidad pero apuntan a que debería ser un proceso más simple. Además varios usuarios fallaron la prueba que consiste en añadir días en los que no podrían acudir al viaje, muy probablemente porque en la aplicación estos días se llaman “eventos” que no encaja correctamente con lo que representan.

7 Conclusiones y futuros cambios

En este capítulo se comentan las conclusiones obtenidas tras la realización del trabajo y las posibles mejoras que podrían realizarse en el futuro.

7.1 Conclusiones

En conclusión, se han cumplido todos los objetivos definidos para el proyecto en el plazo establecido.

El resultado ha sido una aplicación multiplataforma completamente funcional que asiste a sus usuarios en la planificación previa al viaje y durante el mismo.

- El primer paso del proyecto fue la familiarización con las tecnologías necesarias para su desarrollo, particularmente Para ello se siguieron dos cursos en línea orientados al desarrollo de aplicaciones Flutter y a la conexión de estas aplicaciones con un servidor Node.js local. Estos cursos consistieron en el desarrollo guiado de pequeñas aplicaciones que implementaban versiones básicas de las funcionalidades necesarias para el desarrollo del proyecto como la actualización de información en tiempo real y el manejo de estado de la pantalla. Este paso llevó más tiempo de lo esperado debido a la complejidad de los tutoriales pero fueron una buena inversión de tiempo a largo plazo.
- En las etapas iniciales del proyecto, se desarrolló la base de datos relacional con las tablas necesarias para el correcto funcionamiento de las funciones básicas del sistema, esto incluye el inicio de sesión, mostrar los viajes de cada usuario, y los dos primeros tipos de componente. A medida que avanzó el desarrollo se fueron añadiendo las tablas necesarias para el resto de los componentes del sistema. El diseño de la base de datos implicó algunas dificultades debido a la decisión de implementar un modelo de entidad relación extendido pero una vez diseñada no dio problemas.
- El desarrollo del sistema principal de la aplicación fue el objetivo más complicado de implementar ya que incluía conectar la aplicación al servidor, diseñar el proceso para convertir la información recibida a componentes de la interfaz de usuario y el sistema de actualización de pantalla que se usó también en las pantallas de componentes.
- El desarrollo del servidor que conecta la base de datos con la aplicación fue sorprendentemente sencillo, con la ayuda de los tutoriales y documentación en línea puede implementar rápidamente los eventos y funciones necesarias para el funcionamiento del sistema principal y los dos primeros componentes de la aplicación. Llegado este punto la implementación del resto de los componentes fue muy similar y se desarrollaron sin incidencias.
- El último objetivo consistió en la implementación de varios componentes que los usuarios pueden añadir a su viaje para planificar varios aspectos de este. Los componentes desarrollados fueron: Un componente de distribución de habitaciones, un componente de confirmación de asistencia, un componente de lista de la compra, un componente de equipaje grupal, un componente de asignación de tareas, un componente de gestión de gatos y un componente de selección de fechas.

7.2 Futuros cambios

Además de los problemas mencionados en el capítulo anterior, hay una serie de mejoras que me gustaría incorporar en la aplicación.

- Incorporar sistemas de inicio de sesión con Google y Apple para poder incorporar la aplicación a Play Store y la App Store.
- Añadir más componentes a la aplicación, que debería ser fácil debido a su modularidad. Algunos de estos componentes serían un componente de votaciones, un componente de galería o un componente de reparto de asientos en los coches entre otros.
- Añadir la posibilidad de guardar componentes en carpetas para organizar mejor la información.
- Migrar el servidor a un servicio online para no usar mi ordenador personal.
- Diseñar una interfaz específica para ordenadores ya que el código es compatible para buscadores web pero actualmente la interfaz no se adapta correctamente.
- Implementar un sistema de amigos.

8 Análisis del impacto

En este capítulo se analizará el impacto potencial de este TFG en varios contextos.

Desarrollar este proyecto me ha permitido adquirir nuevas habilidades técnicas que me serán de gran utilidad en mi futuro laboral. Además me ha dado la oportunidad de poner en práctica y construir sobre conocimientos adquiridos durante mi formación, especialmente los relacionados con bases de datos, programación orientada a objetos y comunicación mediante sockets.

Considero que a nivel económico esta aplicación tiene potencial ya que cubre una necesidad no satisfecha en el mercado y tiene un gran potencial de monetización. Una posible vía de ingresos podría ser a través de la publicidad. Dada la naturaleza de la aplicación, podrían incluirse anuncios relacionados con la industria del turismo y los viajes, por ejemplo, compañías aéreas, hoteles o empresas de alquiler de coche entre otros.

En relación con los Objetivos de Desarrollo Sostenible (ODS) de las Naciones Unidas, considero que la aplicación tiene potencial para contribuir positivamente al objetivo 8 “Trabajo decente y crecimiento económico” ya que monetizar la aplicación insertando anuncios estaría creando empleo para el desarrollo y mantenimiento de la aplicación, y al mismo tiempo estaría apoyando el crecimiento de la industria del turismo a través de la publicidad.



9 Bibliografia

[1] TripIt:

<https://www.tripit.com/web/free>

[2] Wanderlog:

<https://wanderlog.com/>

[3] SaveTrip:

<https://savetrip.me/>

[4] WhatsApp:

<https://es.wikipedia.org/wiki/WhatsApp>

[5] Google Drive:

<https://www.google.com/drive/>

[6] Trello:

<https://trello.com/>

[7] Tricount:

<https://www.tricount.com/es/>

[8] Splitwise:

<https://www.splitwise.com/>

[9] Figma:

F. Staiano, Designing and Prototyping Interfaces with Figma: Learn essential UX/UI design principles by creating interactive prototypes for mobile, tablet, and desktop. Packt Publishing Ltd, 2022.

[10] Dart:

R. Yao, *DART Programming in 8 Hours, for Beginners, Learn Coding Fast: Dart Quick Start Guide and Exercises*. Independently Published, 2021.

[11] Flutter:

M. L. Napoli, *Beginning Flutter: A Hands On Guide to App Development*. John Wiley & Sons, 2019.

[12] Node.js:

MarkLogic, “MarkLogic Server Node.js Application Developer’s Guide,” 2019.

[13] JavaScript:

A. Purificación Rives, “Formación para el Empleo 40 MANUAL DE JAVASCRIPT.”

[14] MySQL:

P. DuBois, *MySQL*. Pearson Education, 2008.

[15] Socket.IO:

<https://socket.io/docs/v4/>

[16] No IP:

<https://www.noip.com/es-MX/remote-access>

[17] Visual Studio Code:

B. Johnson, *Visual Studio Code: End-to-End Editing and Debugging Tools for Web Developers*. John Wiley & Sons, 2019.

[18] Android Studio:

N. Smyth, *Android Studio 4.2 Development Essentials - Java Edition: Developing Android Apps Using Android Studio 4.2, Java and Android Jetpack*. eBookFrenzy, 2021.

Este documento esta firmado por



Firmante	CN=tfgm.fi.upm.es, OU=CCFI, O=ETS Ingenieros Informaticos - UPM, C=ES
Fecha/Hora	Sun Jul 09 19:37:25 CEST 2023
Emisor del Certificado	EMAILADDRESS=camanager@etsiinf.upm.es, CN=CA ETS Ingenieros Informaticos, O=ETS Ingenieros Informaticos - UPM, C=ES
Numero de Serie	561
Metodo	urn:adobe.com:Adobe.PPKLite:adbe.pkcs7.sha1 (Adobe Signature)